République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Direction Générale des Etudes Technologiques
ISET de Rades
Département Technologies de l'Informatique

الجمهورية التونسية
وزارة التعليم العالي والبحث العلمي
الإدارة العامة للدراسات التكنولوجية
المعهد العالي للدراسات التكنولوجية برادس
قسـم تكنولوجيات الإعلامية

Rapport de Projet de Fin d'Etudes

LICENCE APPLIQUEE EN TECHNOLOGIES DE L'INFORMATIQUE

PARCOURS : *SYSTEMES EMBARQUES ET MOBILES* ***(SEM)***

Système Intelligent de gestion du trafic « Smart Traffic System »

Entreprise : SOCIETE LIFEYE

Réalisé par :
Amen Allah BAYOUDH
Med Aziz TURKI

Encadré par (ISET) :
Mr. Sami MELKI

Code PFE *: SEM-24-07*

Année universitaire *: 2023/2024*

Dédicace

Dédicace de TURKI Med Aziz

Je dédie ce projet à nos parents, qui nous ont soutenus et encouragés tout au long de nos années d'études. Que ce travail soit le témoignage de notre profonde reconnaissance envers eux. À mes frères, qui bien qu'étant loin de nous, n'ont jamais cessé de nous inspirer par leur courage et leur réussite. Votre soutien à distance a été une source constante de motivation. À toute la famille de mon binôme, qui nous a également soutenus avec chaleur et encouragements. À nos professeurs, dont la guidance et le soutien ont été essentiels pour surmonter les obstacles et trouver des solutions tout au long de ce projet. Leur dévouement et leur expertise nous ont aidés à réussir.

Enfin, à nous-mêmes. Nous avons fait preuve de persévérance, de détermination, et de résilience tout au long de cette étape cruciale de notre vie. Malgré les défis et les obstacles, nous avons relevé la tête et avancé avec ténacité. Cette dédicace est une célébration de notre travail acharné, de nos réalisations, et de notre capacité à surmonter les difficultés. Nous sommes fiers de ce que nous avons accompli et nous nous souviendrons toujours de cette réussite.

Dédicace de BAYOUDH Amen Allah

Je dédie ce projet à ma famille entière : mes parents, mon frère, ma sœur, ma grand-mère, mes oncles et tantes. Chacun a contribué à ma réussite par son amour, son soutien, et ses encouragements constants. Que ce travail soit le témoignage de ma profonde reconnaissance envers tous. À nos professeurs, dont la guidance et le soutien ont été essentiels pour surmonter les obstacles et trouver des solutions tout au long de ce projet. Leur dévouement et leur expertise nous ont aidés à réussir.

Enfin, à nous-mêmes. Nous avons fait preuve de persévérance, de détermination, et de résilience tout au long de cette étape cruciale de notre vie. Malgré les défis et les obstacles, nous avons relevé la tête et avancé avec ténacité. Cette dédicace est une célébration de notre travail acharné, de nos réalisations, et de notre capacité à surmonter les difficultés. Nous sommes fiers de ce que nous avons accompli et nous nous souviendrons toujours de cette réussite.

Remerciements

Ce projet n'aurait pu être mené à bien sans l'aide précieuse que nous avons reçue, tant sur le plan moral que technique. C'est donc avec grand honneur que nous dédions ces lignes à toutes les personnes qui, de près ou de loin, ont été à nos côtés et ont contribué à l'élaboration de ce projet de fin d'études.

Nos remerciements les plus sincères s'adressent à :

M. Sami Melki, notre encadrant, pour son accompagnement et ses précieux conseils. Sa disponibilité et sa confiance nous ont été d'une grande aide. Ses encouragements constants ont été essentiels pour la réussite de ce travail.

Nous souhaitons également exprimer notre gratitude envers nos familles, qui nous ont permis de poursuivre de longues études et nous ont soutenus tout au long de ces trois années.

À tous ceux qui ont contribué à cette réalisation, nous adressons notre profonde reconnaissance.

Table des matières

Introduction générale 1

Chapitre 1 : Présentation du cadre de projet 3

Introduction 4

1. Présentation de l'organisme 4

1.1. Présentation de la société 4

1.2. Organigramme de la société 4

2. Etude de l'existant 5

2.1. Description de l'existant 5

2.2. Critique de l'existant 6

2.3. Solution proposée 6

3. Planning prévisionnel 8

Conclusion 8

Chapitre 2 : Revue de la littérature 9

Introduction 10

1. Définition de la ville intelligente 10

2. Les technologies IoT au cœur des villes intelligentes 11

3. Les usages clés des technologies IoT pour la ville intelligente 11

3.1. Gestion du trafic, du transport et des solutions de stationnement 11

3.2. Exemples de villes européennes utilisant ces technologies 12

4. L'IA et les villes intelligentes 12

5. La communication dans la ville intelligente (LoRa) 13

Conclusion 13

Chapitre 3 : Spécification des besoins 14

Introduction 15

1. Besoins fonctionnels 15

2. Besoins non-fonctionnels 15

Conclusion 16

Chapitre 4 : Conception du système 17

Introduction 18

1. Conception du carrefour 18

1.1. Choix du carrefour 18

1.2. Cas neutre 20

1.3. Cas d'un véhicule prioritaire 20

1.4. Cas d'un véhicule intelligent 21

2. Choix des Matériaux 22

2.1. Raspberry Pi 4 (4Go RAM) 22
2.2. Webcams pour Raspberry Pi (8 MP) 23
2.3. LED de Signalisation Préfabriquées 23
2.4. Carte RFID et Lecteur RFID (RC522) 24
2.5. Heltec Wireless Stick 25
2.6. Câbles Jumper 25
3. Choix des technologies 26
3.1. LoRa 26
3.2. Protocole MQTT 27
3.3. HiveMQ 28
3.4. Modèle YOLOv8n 28
4. Ecosystème 29
4.1. Schéma 29
4.2. Partie gestion des voitures prioritaires 31
4.3. Partie notification des véhicules intelligents 31
5. Schémas et montages 32
5.1. Connexion du Raspberry Pi 4 32
5.3. Connexion du Heltec Wireless Stick (récepteur LoRa) 33
5.2. Connexion du Heltec Wireless Stick (émetteur LoRa) 34
Conclusion 34
Chapitre 5 : Réalisation 35
Introduction 36
1. Installation de Raspbian sur Raspberry Pi 36
1.1. Téléchargement et Installation de SD Card Formatter 36
1.2. Formatage de la Carte SD 37
1.3. Téléchargement de Berryboot 38
1.4. Copie de Berryboot sur la Carte SD 38
1.5. Insertion de la Carte SD dans le Raspberry Pi 38
1.6. Initialisation de Berryboot 38
2. Configuration de VNC sur Raspberry Pi 40
2.1. Installation de RealVNC sur Windows 40
2.2. Activation du serveur VNC 40
2.3. Configuration du réseau 41
2.4. Connexion depuis un autre appareil 41
3. Configuration des Caméras 42
3.1. Connexion des caméras 42
3.2. Installation des bibliothèques nécessaires 42

3.3. Identification des indices des caméras 42
4. Étiquetage des Images pour l'Entraînement du Modèle YOLOv8 42
4.1. Installation de LabelImg 42
4.2. Ouverture de LabelImg 43
4.3. Étiquetage des images 43
4.4. Sauvegarde des annotations 44
5. Entraînement du Modèle YOLOv8 sur VS Code 44
5.1. Préparation de l'environnement 44
5.2. Chargement des données d'entraînement 44
5.3. Script d'entraînement 45
5.4. Visualisation de l'entraînement 45
5.5. Validation du modèle 45
5.6. Analyse des résultats 46
6. Construction du Maquette 48
6.1. Réalisation de l'image de l'intersection et impression sur contreplaqué 48
6.2. Pose des poteaux de feux au carrefour 49
6.3. Placement des caméras et des feux de signalisation préfabriqués dans chaque voie 49
7. Construction de la voiture intelligente 50
7.1. Solution Choisie : Intégration du système LoRa avec RFID 50
7.2. Prototype de la voiture de test 51
8. Défis Majeurs Rencontrés et Solutions 52
8.1. Délai de Détection avec la Raspberry Pi 52
8.2. Gestion de la Connexion entre les Codes 52
8.3. Problèmes avec l'Intégration de HeltecWirelessStick et du Lecteur RFID 53
9. Test Final 54
9.1. Cas neutre 54
9.2. Cas d'un véhicule prioritaire (ambulance) 55
9.3. Cas d'un véhicule intelligent 56
Conclusion 58
Conclusion générale 59
Nétographie 60
ANNEXES 62
1. Logiciels 62
2. Langages de programmation 65
3. Librairies Python 66
4. Librairies Arduino 67

Liste des figures

Figure 1: Organigramme de la société 5
Figure 2: Ville intelligente 10
Figure 3: Le Carrefour à Quatre Voies 18
Figure 4: L'Intersection en T 19
Figure 5: Cas neutre 20
Figure 6: Cas d'un véhicule prioritaire 20
Figure 7: Cas d'un véhicule intelligent avant l'intersection 21
Figure 8: Cas d'un véhicule intelligent après l'intersection 21
Figure 9: Raspberry Pi 4 22
Figure 10: Webcam pour Raspberry Pi 23
Figure 11: LED de Signalisation Préfabriquées 23
Figure 12: Carte RFID et Lecteur RFID 24
Figure 13: Heltec Wireless Stick 25
Figure 14: Câbles Jumper 25
Figure 15: LoRa 26
Figure 16: MQTT 27
Figure 17: HiveMQ 28
Figure 18: YOLOv8 28
Figure 19: Connexion du Raspberry Pi 4 32
Figure 20: Connexion du Heltec Wireless Stick (récepteur LoRa) 33
Figure 21: Connexion du Heltec Wireless Stick (émetteur LoRa) 34
Figure 22: Raspbian 36
Figure 23: Téléchargement de SD Card Formatter 36
Figure 24: Installation du SD Card Formatter 37
Figure 25: Formatage de la Carte SD 37
Figure 26: Téléchargement de Berryboot 38
Figure 27: Extraction de contenu du fichier téléchargé 38
Figure 28: Etape 1 d'initialisation de Berryboot 39
Figure 29: Etape 2 d'initialisation de Berryboot 39
Figure 30: Etape 3 d'initialisation de Berryboot 39
Figure 31: Etape 4 d'initialisation de Berryboot 40
Figure 32: Installation de RealVNC 40
Figure 33: Activation du serveur VNC 40
Figure 34: Configuration d'adresse IP (statique) 41
Figure 35: Insertion d'adresse IP du Raspberry sur le VNC 41
Figure 36 : Authentification du VNC 41
Figure 37: Connexion des caméras 42
Figure 38: Installation d'OpenCV 42
Figure 39: Installation de LabelImg 43
Figure 40: Ouverture de LabelImg 43
Figure 41: Étiquetage des images 43
Figure 42: Configuration de VS Code avec l'extension Python 44
Figure 43: Chargement des données d'entraînement 44
Figure 44: Visualisation de l'entraînement 45
Figure 45: Validation du modèle 45
Figure 46: Modèle entraîné 45
Figure 47: Résultat du modèle 46
Figure 48: Confusion Matrix 47
Figure 49: F1-Confidence Curve 47
Figure 50: Maquette 48
Figure 51: Maquette vue de dessous 48
Figure 52: Emplacement des poteaux 49

Figure 53: Placement des caméras et des feux 49
Figure 54: Prototype de la voiture de test 51
Figure 55: Vue de dessous de la voiture de test..... 51
Figure 56: État normal du feu..... 54
Figure 57: Affichage de l'état + temps 54
Figure 58: Ambulance détectée 55
Figure 59: Affichage d'urgence 55
Figure 60: Envoi du feu rouge..... 56
Figure 61: Envoi du feu jaune 56
Figure 62: Envoi du feu vert..... 57
Figure 63: Arrivée à l'intersection 57
Figure 64: VS Code..... 62
Figure 65: PlatformIO 62
Figure 66: Arduino IDE 63
Figure 67: Thonny..... 63
Figure 68: MS Paint 64
Figure 69: Fritzing 64
Figure 70: LabelImg..... 64
Figure 71: Python..... 65
Figure 72: C++/Arduino..... 65

Liste des Tableaux

Tableau 1: Planning prévisionnel 8

Introduction générale

Selon l'organisation mondiale de la santé (OMS), environ 40% des accidents de la route dans le monde surviennent dans les carrefours.

Les systèmes de gestion de trafic routier intelligents renforcent considérablement la sécurité sur les routes grâce à leur capacité à surveiller en temps réel les conditions de circulation, à détecter les incidents et à réagir rapidement pour minimiser les risques d'accidents. En utilisant des technologies telles que l'Internet des Objets (IoT), les capteurs et la surveillance vidéo, ces systèmes peuvent identifier les zones à risque, contrôler les feux de signalisation de manière adaptative en fonction du trafic en temps réel et prioriser les véhicules d'urgence. De plus, en collectant et en analysant des données sur les schémas de circulation et les comportements des conducteurs, ces systèmes permettent une planification proactive des mesures de sécurité routière, telles que l'installation de dispositifs de sécurité supplémentaires et l'optimisation de la conception des routes. En conséquence, les systèmes de gestion de trafic routier intelligents contribuent à réduire les accidents, à sauver des vies et à améliorer la sécurité pour tous les usagers de la route.

Dans le contexte de l'urbanisation croissante et de la densification des villes, la gestion du trafic routier devient un enjeu majeur pour les municipalités. Les systèmes de gestion de trafic routière intelligents (SGTRI) représentent une solution technologique prometteuse pour répondre à ces défis. Ces systèmes visent à optimiser la fluidité du trafic, à améliorer la sécurité routière et à réduire l'impact environnemental des transports urbains.

Dans les villes intelligentes, les systèmes de gestion de carrefour équipés de l'IoT et de la surveillance transforment la mobilité urbaine. Ils réduisent la congestion en adaptant dynamiquement les feux de circulation, ce qui fluidifie le trafic et diminue les temps d'attente. La sécurité est renforcée par une meilleure régulation du flux des véhicules et la priorisation des services d'urgence, réduisant ainsi les accidents et améliorant les interventions. L'efficacité énergétique est accrue grâce à la diminution des arrêts inutiles, entraînant une baisse de la consommation de carburant et des émissions polluantes. La gestion proactive des incidents est possible grâce à la détection immédiate des anomalies, permettant des réactions rapides et efficaces. La collecte de données joue un rôle crucial dans l'aménagement et la gestion des infrastructures, tandis que l'intégration avec d'autres services de la ville intelligente crée un écosystème urbain cohérent et optimisé. Ces systèmes sont le cœur battant des villes de demain, où technologie et durabilité se rencontrent pour un avenir plus vert et connecté.

Ce rapport est organisé en cinq chapitres comme suit :

Le premier chapitre présente le cadre du projet. Le deuxième chapitre est consacré à la revue de la littérature. Le troisième chapitre spécifie les besoins du système. Le quatrième chapitre aborde la conception du système. Enfin, le cinquième chapitre traite de la réalisation du projet.

Chapitre 1 :
Présentation du cadre de projet

Introduction

Ce chapitre présente une analyse des systèmes actuels de gestion du trafic urbain et propose des améliorations innovantes. Après une brève introduction de MooMe, nous explorerons les limitations des systèmes existants et décrirons une solution avancée pour optimiser la fluidité et la sécurité du trafic.

1. Présentation de l'organisme

1.1. Présentation de la société

Fondée en 2016, la start-up tunisienne MooMe se spécialise dans la création et la commercialisation d'applications mobiles innovantes visant à simplifier le quotidien de ses utilisateurs. Cette entreprise a été fondée par un groupe d'entrepreneurs experts et passionnés par les nouvelles technologies et l'innovation. Il nous est donc agréable de vous présenter les objectifs de notre entreprise :

- Améliorer l'efficacité et la productivité des technologies en proposant des solutions innovantes
- Offrir des produits de qualité supérieure aux consommateurs en garantissant la traçabilité et la transparence des produits
- Fournir des services de qualité supérieure pour répondre aux besoins des clients et des utilisateurs.
- Être à la pointe de l'innovation dans le domaine des technologies mobiles en Tunisie

1.2. Organigramme de la société

Nous avons l'honneur de vous présenter notre équipe de professionnels dédiés à la réussite de notre entreprise. Composée d'experts passionnés, cette équipe dirigeante est formée d'un CEO, d'un CFO et d'un CTO, chacun apportant son expertise dans leur domaine respectif. Des responsables de marketing, de vente et de produit sont déterminés à offrir aux clients des solutions innovantes pour répondre à leurs besoins. Enfin, des spécialistes en intelligence artificielle mettent leur expertise en matière de technologies de pointe au service des projets ambitieux.[1]



Figure 1: Organigramme de la société

2. Etude de l'existant

2.1. Description de l'existant

Actuellement, plusieurs villes à travers le monde ont adopté des Systèmes Intelligents de Gestion du Trafic (**SIGT**) pour améliorer la fluidité du trafic et la sécurité. Deux exemples notables sont le système de gestion du trafic **SCATS** (Sydney Coordinated Adaptive Traffic System) utilisé en Australie et le système **SCOOT** (Split Cycle Offset Optimization Technique) utilisé au Royaume-Uni.[2] [3]

SCATS est un système en temps réel qui ajuste les feux de signalisation en fonction des conditions de trafic détectées par des capteurs sur les routes. Ce système permet de minimiser les temps d'attente aux feux et de réduire la congestion en s'adaptant dynamiquement aux variations du trafic. [2] [3]

SCOOT, de son côté, utilise des capteurs pour collecter des données sur le flux de trafic et optimise les cycles des feux de signalisation pour réduire les retards et améliorer la circulation. **SCOOT** peut s'adapter aux changements du trafic en temps réel, offrant ainsi une solution flexible et efficace pour la gestion urbaine du trafic. [2] [3]

2.2. Critique de l'existant

Malgré les avancées significatives apportées par des systèmes comme **SCATS** et **SCOOT**, plusieurs insuffisances subsistent, surtout en ce qui concerne la gestion des véhicules prioritaires. Les insuffisances du système actuel, en comparaison avec notre solution proposée, sont multiples :

- **Réactivité limitée pour les véhicules prioritaires** : Bien que **SCATS** et **SCOOT** puissent ajuster les feux de signalisation en fonction du trafic général, ils ne sont pas spécifiquement optimisés pour détecter et accorder la priorité aux véhicules d'urgence en temps réel. Les véhicules prioritaires doivent souvent se fier aux sirènes et aux feux clignotants pour naviguer dans le trafic, ce qui peut entraîner des délais. [2] [3]

- **Manque de coordination** : Les systèmes existants ne coordonnent pas les feux de signalisation spécifiquement pour les véhicules d'urgence, ce qui peut nécessiter des interventions manuelles de la part des agents de circulation pour faciliter leur passage. [2] [3]

- **Risque accru d'accidents** : En l'absence de coordination intelligente, les véhicules prioritaires peuvent rencontrer des obstacles inattendus ou des conducteurs imprudents, augmentant le risque d'accidents. [2] [3]

- **Absence de surveillance en temps réel** : Les systèmes comme **SCATS** et **SCOOT** manquent de capacités de surveillance et d'adaptation en temps réel spécifiquement pour les véhicules d'urgence, ce qui limite leur efficacité en situation d'urgence. [2] [3]

2.3. Solution proposée

Pour pallier les insuffisances mentionnées ci-dessus, nous proposons de développer un ***Système Intelligent de gestion du trafic*** avec les composantes suivantes :

1. Reconnaissance des Véhicules Prioritaires :

- Utilisation de caméras équipées de technologies de reconnaissance d'image pour détecter les véhicules prioritaires en temps réel.
- Identification rapide et précise des véhicules d'urgence afin de leur accorder la priorité sur les routes.

2. Adaptation Dynamique du Trafic :
   - Implémentation d'algorithmes avancés qui ajustent les feux de signalisation en fonction de la présence et du trajet des véhicules prioritaires.
   - Coordination des feux pour garantir un passage fluide et sécurisé des véhicules d'urgence, réduisant ainsi les temps d'intervention.
3. Tableau de Bord pour Véhicules Intelligents :
   - Intégration d'un système qui permet aux véhicules intelligents de recevoir en temps réel des informations sur l'état des feux de signalisation (rouge, jaune, vert) de la voie qu'ils empruntent.
   - Cette fonctionnalité permet aux conducteurs de ces véhicules d'anticiper les changements de signalisation et d'adapter leur conduite en conséquence, améliorant ainsi la fluidité et la sécurité du trafic.

Avantages de la Solution Proposée :

- **Efficacité Accrue** : Réduction significative des délais pour les véhicules prioritaires grâce à une gestion intelligente et coordonnée du trafic.
- **Sécurité Améliorée** : Moins de risques d'accidents grâce à une meilleure coordination des feux de signalisation et une information en temps réel sur l'état des feux pour les véhicules intelligents.
- **Adaptation et Évolutivité** : Système capable de s'adapter aux changements des conditions de trafic et de s'intégrer facilement aux infrastructures existantes.

3. Planning prévisionnel

Tableau 1: Planning prévisionnel

Etape \ Semaine	Février				Mars				Avril				Mai			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Etude préalable			✖	✖	✖											
Conception							✖	✖	✖	✖						
Réalisation											✖	✖	✖	✖		
Test et Validation															✖	✖

Conclusion

Ce chapitre a identifié les insuffisances des systèmes actuels de gestion du trafic, notamment leur réactivité limitée et le manque de coordination. La solution proposée utilise des technologies de reconnaissance d'image et des algorithmes adaptatifs pour optimiser la gestion du trafic, améliorant ainsi l'efficacité et la sécurité urbaines, tout en répondant aux objectifs de qualité et d'innovation de MooMe.

Chapitre 2 : Revue de la littérature

Introduction

Avec l'urbanisation rapide, les villes intelligentes émergent comme une solution innovante pour améliorer la qualité de vie des citadins. Une ville intelligente utilise des technologies avancées, notamment l'Internet des objets (IoT) et l'intelligence artificielle (IA), pour optimiser la gestion urbaine. Ce chapitre explore les composantes clés des villes intelligentes, le rôle crucial des technologies IoT et IA, et l'importance des solutions de communication comme LoRa. Nous verrons comment ces technologies améliorent la gestion du trafic, l'optimisation énergétique et les services urbains.

1. Définition de la ville intelligente

Une smart city, ou ville intelligente, est une ville où la plupart des objets urbains sont connectés. Une telle ville s'appuie sur une multitude de capteurs et transmetteurs de faible puissance afin de maintenir une connexion permanente. Les données collectées en temps réel sont traitées par une intelligence informatique qui optimise le fonctionnement des éléments de la ville. L'objectif affiché est d'améliorer la vie des citadins en rendant leur quotidien plus agréable et efficace. Une étude récente montre que les villes intelligentes peuvent économiser $ 5 trillions de coûts par 2022. Une infrastructure urbaine intelligente incorpore divers facteurs : les technologies de l'information et de la communication (TIC), l'Internet des objets (IoT), les partenariats public-privé, le capital social et humain. [4][5][6]



Figure 2: Ville intelligente

2. Les technologies IoT au cœur des villes intelligentes

L'Internet des objets (IoT) relie les appareils électroniques (autres que les ordinateurs et les smartphones) à Internet pour un suivi et une gestion efficace des activités quotidiennes. Dans le domaine du développement urbain, les technologies de l'information et de la communication (TIC) et l'IoT sont des éléments constitutifs essentiels de la création d'une infrastructure intelligente pour administrer, servir et accompagner la population urbaine en constante augmentation.

Une ville intelligente nécessite une efficacité technologique dans divers domaines, notamment :

- Les transports et la mobilité
- La communication
- Les services
- La sécurité
- La relation citoyenne

Les applications IoT permettent notamment d'optimiser :

- La performance des bâtiments
- Les consommations d'énergie
- La gestion du mobilier urbain
- L'élimination des déchets

Elles facilitent également la mobilité. Les bénéficiaires de ces dispositifs sont les consommateurs, les citoyens, les institutions territoriales et les entreprises privées.[4]

3. Les usages clés des technologies IoT pour la ville intelligente

3.1. Gestion du trafic, du transport et des solutions de stationnement

Les problèmes de mobilité sont une question majeure pour les citadins. Les infrastructures IoT fournissent des solutions dynamiques et intelligentes pour réduire les problèmes de mobilité et de stationnement en :

- Évitant les embouteillages

- Suggérant le temps optimal pour se déplacer
- Fournissant des informations sur les places de stationnement disponibles
- Ces technologies visent à économiser du temps, de l'énergie, des émissions de gaz et particules, et à maintenir un flux de circulation fluide. [4]

3.2. Exemples de villes européennes utilisant ces technologies

Paris a minimisé son problème de trafic en adoptant des détecteurs de stationnement.

Londres travaille sur un projet de stationnement intelligent qui aide les conducteurs à localiser des places de stationnement libres. Ces villes expérimentent également des modèles de partage de véhicules et de vélos électriques.[4]

4. L'IA et les villes intelligentes

L'intelligence artificielle (IA) permet aux ordinateurs de réaliser des tâches complexes comme la reconnaissance vocale, la vision par ordinateur et le traitement du langage. Elle utilise des techniques telles que l'apprentissage automatique pour apprendre à partir de données et agir sans instructions explicites. L'IA peut être basée sur des règles ou des modèles statistiques, avec l'apprentissage automatique étant le plus courant aujourd'hui. L'IA offre de nombreux avantages pour la gestion et la planification urbaine. Elle permet d'analyser et de simuler des systèmes urbains complexes, rendant les villes plus durables et agréables à vivre. L'IA peut :

- Améliorer l'utilisation des ressources
- Réduire les embouteillages
- Optimiser la consommation d'énergie
- Promouvoir la réduction des déchets
- Offrir des services personnalisés aux citoyens

Elle aide également à détecter les menaces pour la sécurité et à fournir des recommandations personnalisées. Cependant, l'IA doit être mise en œuvre de manière responsable, en respectant la transparence et la protection de la vie privée.[7]

5. La communication dans la ville intelligente (LoRa)

LoRa, ou Long Range, révolutionne la communication sans fil avec ses puces à faible consommation d'énergie, permettant des années de fonctionnement sur batterie. Contrairement à la 4G ou la 5G, LoRa opère dans des bandes non réglementées, idéales pour les villes intelligentes comme Boston. Les capteurs intégrés aux lampadaires collectent des données via LoRa, offrant des solutions pour le stationnement, la circulation, la maintenance et la gestion des déchets. Son intégration avec les LED, comme à Tallinn, promet une efficacité et une sécurité accrues. Avec LoRa, les villes économisent sur l'infrastructure et la maintenance, stimulant l'innovation urbaine. Grâce à sa portée impressionnante, pouvant atteindre jusqu'à 50 km en milieu rural ou sur plusieurs kilomètres en milieu urbain dense, LoRa garantit une connectivité fiable et étendue pour les applications urbaines.[6]

Conclusion

En résumé, les villes intelligentes, grâce à l'intégration des technologies IoT et de l'IA, offrent des solutions innovantes pour améliorer la gestion urbaine et la qualité de vie des citoyens. Ces technologies optimisent les transports, la consommation d'énergie, la gestion des déchets et la sécurité urbaine. De plus, des solutions comme LoRa permettent une communication sans fil efficace et économique, essentielle pour la gestion des infrastructures urbaines. Avec une adoption stratégique et éthique, les villes intelligentes promettent un avenir urbain plus durable, efficace et agréable pour tous.

Chapitre 3 :

Spécification des besoins

Introduction

La gestion efficace du trafic est essentielle pour les villes modernes. Ce chapitre explore les besoins cruciaux pour un système capable de reconnaître les véhicules prioritaires, d'ajuster les feux de circulation en temps réel et de garantir une performance fiable et sécurisée. Ces exigences sont essentielles pour créer un réseau de transport intelligent et adaptable.

1. Besoins fonctionnels

Les besoins fonctionnels définissent les fonctionnalités spécifiques que le système doit accomplir pour répondre aux attentes des utilisateurs et atteindre les objectifs fixés. Ils décrivent ce que le système doit faire, les actions qu'il doit pouvoir exécuter et les services qu'il doit offrir.

Gestion des véhicules prioritaire :

- **Reconnaissance des véhicules prioritaires** : Utilisation de caméras équipées de technologies de reconnaissance d'image pour identifier les véhicules prioritaires tels que les ambulances, les voitures de police, etc.
- **Adaptation du trafic** : Modification dynamique des feux de circulation pour donner la priorité aux véhicules identifiés, garantissant ainsi leur passage fluide et rapide.
- **Envoi de notifications aux véhicules intelligents** : Transmission en temps réel des informations sur l'état des feux de circulation aux véhicules intelligents, leur permettant d'adapter leur conduite en conséquence.

2. Besoins non-fonctionnels

Les besoins non-fonctionnels concernent les critères de qualité et les caractéristiques du système. Ils ne définissent pas les fonctionnalités spécifiques, mais décrivent les attentes en termes de performance, de fiabilité, de sécurité et de maintenance. Ils sont essentiels pour garantir que le système est utilisable et efficace dans des conditions réelles.

1. **Performances**

- **Réponse rapide** : Le système doit être capable de réagir rapidement aux changements de trafic, minimisant ainsi les délais et assurant une fluidité optimale.

2. **Fiabilité**

- **Robustesse et tolérance aux pannes** : Le système doit être conçu pour être robuste et capable de fonctionner de manière continue, même en cas de défaillance partielle.

3. **Sécurité**

- **Protection des données** : Assurer la sécurité des données transmises et stockées, ainsi que la prévention des accès non autorisés.

4. **Extensibilité**

- **Capacité d'évolution** : Le système doit être flexible et évolutif, permettant l'ajout de nouvelles fonctionnalités et l'adaptation à de nouveaux types de véhicules prioritaires à l'avenir.

5. **Affichage du temps restant**

- **Informations claires et précises** : Affichage en temps réel du temps restant pour chaque feu de circulation et l'état de chaque voie, permettant aux conducteurs de mieux anticiper les changements.

Conclusion

Les besoins fonctionnels et non-fonctionnels sont fondamentaux pour la conception d'un système de gestion du trafic intelligent et efficace. Les besoins fonctionnels garantissent que le système répond aux exigences spécifiques des utilisateurs, notamment en ce qui concerne la reconnaissance et la gestion des véhicules prioritaires. Les besoins non-fonctionnels, quant à eux, assurent que le système est performant, fiable, sécurisé et capable de s'adapter aux évolutions futures. En combinant ces deux types de besoins, nous posons les bases d'un système robuste et évolutif, capable de relever les défis de la gestion du trafic urbain moderne.

Chapitre 4 : Conception du système

Introduction

Dans ce chapitre, nous approfondissons la conception détaillée de notre système de contrôle de trafic intelligent. Nous examinerons chaque composant en détail, en discutant de leurs informations générales, avantages, inconvénients, et des raisons pour lesquelles nous les avons choisis pour notre projet. Notre analyse commence par le choix des types d'intersections, mettant en lumière la complexité et les défis associés à la gestion du flux de trafic aux carrefours à quatre voies et aux intersections en T. Cette exploration approfondie vise à fournir une compréhension complète de l'architecture du système et de son fonctionnement dans des scénarios de trafic urbain réels.

1. Conception du carrefour

1.1. Choix du carrefour

Dans ce chapitre, nous allons examiner en détail chaque composant de notre système de contrôle de trafic intelligent. Chaque section inclura des informations générales, les avantages et inconvénients, et les raisons pour lesquelles nous avons choisi chaque composant pour notre projet.

Le Carrefour à Quatre Voies est une intersection complexe où deux routes se croisent, formant une jonction avec quatre voies de circulation, régulée par des feux de signalisation ou des panneaux d'arrêt. Ce type d'intersection permet une gestion sophistiquée de la circulation dans les quatre directions, représentant des scénarios réalistes de trafic urbain. Cependant, cela entraîne une complexité accrue dans la synchronisation des feux et la gestion des flux de trafic, augmentant le risque d'accidents en cas de mauvaise gestion. Pour couvrir efficacement le flux de circulation dans toutes les directions, quatre caméras sont positionnées stratégiquement.

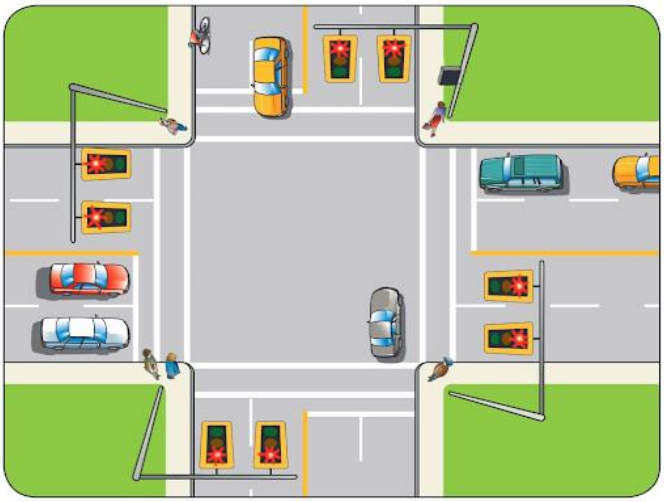


Figure 3: Le Carrefour à Quatre Voies

L'Intersection en T est formée par une route traversant une autre à angle droit, créant une forme en T, et est souvent régulée par des panneaux d'arrêt ou des feux de signalisation. Ce type d'intersection est relativement plus simple à modéliser et offre une représentation réaliste des scénarios courants de trafic urbain. Néanmoins, la gestion du trafic peut devenir plus complexe en cas de forte densité, nécessitant une synchronisation précise des feux et une gestion des priorités de passage. En général, trois caméras sont positionnées sur chaque voie pour couvrir efficacement la route principale et la route se terminant.

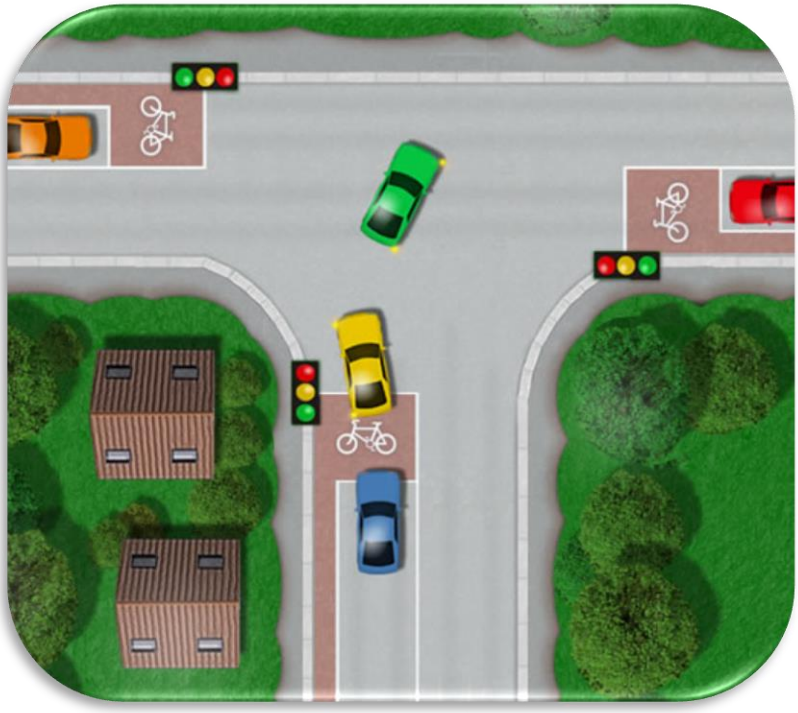


Figure 4: L'Intersection en T

Nous avons choisi de modéliser une intersection en T plutôt qu'un carrefour à quatre voies pour plusieurs raisons stratégiques et pratiques. L'intersection en T, bien que courante dans les environnements urbains, présente une structure moins complexe et plus facile à gérer pour un système de contrôle de trafic intelligent. La simplicité relative de l'intersection en T permet de concentrer nos efforts sur l'optimisation des algorithmes de détection et de priorisation des véhicules d'urgence sans être submergés par la complexité accrue d'une gestion à quatre voies. De plus, l'intersection en T représente une configuration urbaine typique, permettant de tester notre système dans des conditions réalistes tout en offrant une clarté dans la synchronisation des feux de signalisation et la gestion des priorités de passage. En choisissant ce type d'intersection, nous visons à démontrer les capacités et l'efficacité de notre système de manière plus contrôlée et significative, tout en réduisant les risques et les variables liées à la complexité d'un carrefour à quatre voies.

D’après ce choix en on va décrire ce qui va se produire dans chacun de ces scénarios.

1.2. Cas neutre

Dans le cas neutre, c'est-à-dire sans véhicules exceptionnels, le flux suivra le système de circulation régulier : chaque voie aura son tour avec le feu vert qui durera 7 secondes (nous avons choisi cette durée car il s'agit d'un prototype, ce qui permet de montrer le changement de flux beaucoup plus rapidement). Avant que le feu vert ne s'allume, le feu jaune sera allumé pendant 3 secondes. Et, les feux rouges seront allumés dans les autres voies quand ce n'est pas leur tour.

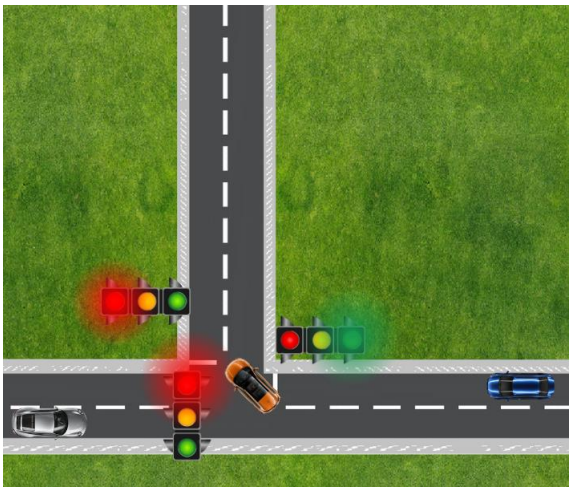


Figure 5: Cas neutre

1.3. Cas d'un véhicule prioritaire

Dans le cas d'un véhicule prioritaire, c'est-à-dire lorsqu'une ambulance, une voiture de police ou tout autre véhicule d'urgence arrive au carrefour, il sera détecté par la caméra placée sur son trajet. En conséquence, le feu vert de cette voie de circulation s'allumera pour lui donner la priorité de passage. Et, les feux rouges seront allumés dans les autres voies.

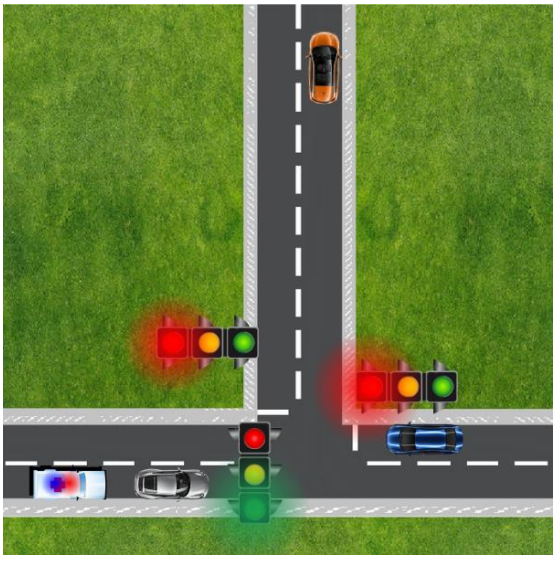


Figure 6: Cas d'un véhicule prioritaire

1.4. Cas d’un véhicule intelligent

Dans le cas d'un véhicule intelligent, c'est-à-dire lorsqu'un véhicule intelligent se dirige vers le carrefour, il détectera l'ID de la voie qu'il va rencontrer, ce qui aura pour effet d'allumer les LED pour correspondre à l'état actuel des feux de circulation de cette voie en temps réel, jusqu'à ce que le véhicule ait franchi l'intersection.

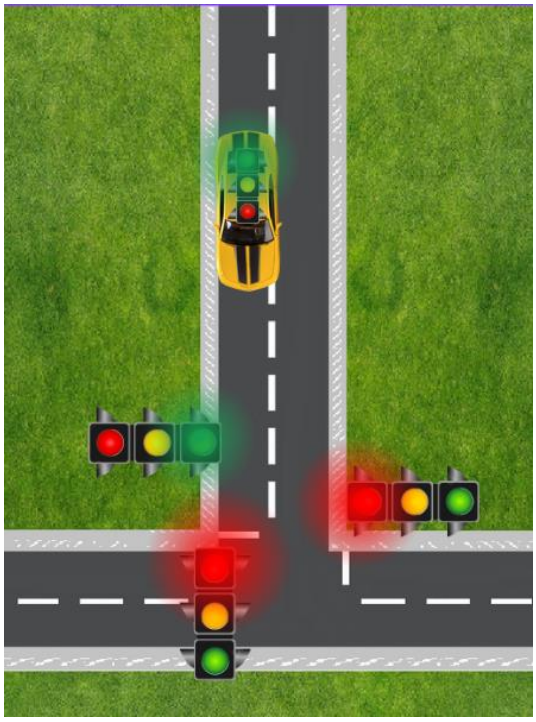


Figure 7: Cas d’un véhicule intelligent avant l'intersection

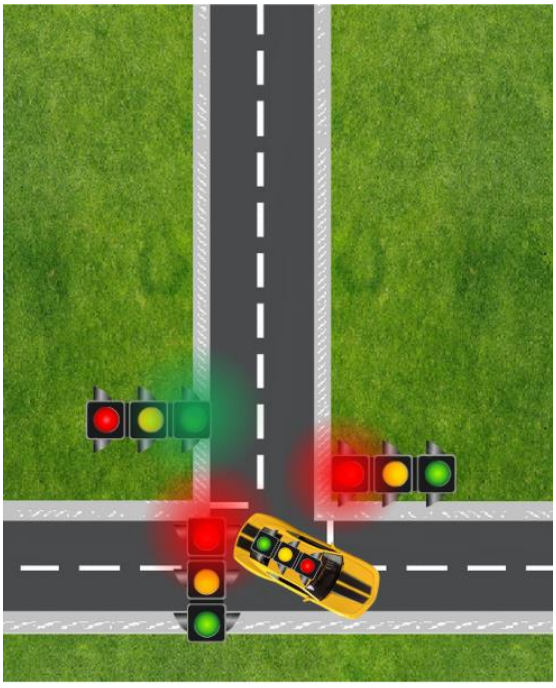


Figure 8: Cas d’un véhicule intelligent après l'intersection

2. Choix des Matériaux

2.1. Raspberry Pi 4 (4Go RAM)



Figure 9: Raspberry Pi 4

Informations Générales : Le Raspberry Pi est un ordinateur monocarte puissant, compact et abordable, largement utilisé dans les projets de prototypage et les applications IoT (Internet of Things). Il fonctionne sous divers systèmes d'exploitation, notamment Raspbian, et dispose de nombreuses interfaces pour connecter des périphériques externes tels que des caméras, des capteurs et des modules de communication.[8][9]

Avantages : Le Raspberry Pi offre une puissance de calcul suffisante pour exécuter des algorithmes complexes, une flexibilité d'utilisation grâce à ses nombreuses interfaces, et une large communauté de support qui facilite le développement et le dépannage. [8][9]

Inconvénients : Par rapport à des microcontrôleurs plus simples, le Raspberry Pi consomme plus d'énergie et peut nécessiter une gestion thermique. De plus, son utilisation peut être excessivement complexe pour des tâches très spécifiques nécessitant une faible consommation d'énergie. [8][9]

Pourquoi ce choix : Le Raspberry Pi a été choisi pour sa capacité à traiter des algorithmes de vision par ordinateur en temps réel et à gérer des tâches de communication complexe, ce qui est essentiel pour notre système de contrôle de trafic intelligent.

2.2. Webcams pour Raspberry Pi (8 MP)



Figure 10: Webcam pour Raspberry Pi

Informations Générales : Les webcams connectées au Raspberry Pi sont utilisées pour capturer des flux vidéo en temps réel des intersections de trafic. Ces caméras sont essentielles pour la reconnaissance d'images et la détection des véhicules d'urgence. [8]

Avantages : Les webcams modernes offrent une haute résolution, une bonne performance dans diverses conditions de lumière, et sont relativement abordables. Elles sont également faciles à intégrer avec le Raspberry Pi grâce à des interfaces standardisées. [8]

Inconvénients : Les webcams nécessitent une gestion appropriée de la lumière et de la position pour capturer des images de haute qualité. Elles peuvent également générer une grande quantité de données, nécessitant des capacités de traitement et de stockage adéquates. [8]

Pourquoi ce choix : Nous avons choisi les webcams pour leur capacité à fournir des flux vidéo de haute qualité, indispensables pour les algorithmes de vision par ordinateur utilisés dans notre système de détection des véhicules d'urgence.

2.3. LED de Signalisation Préfabriquées



Figure 11: LED de Signalisation Préfabriquées

Informations Générales : Les LED de signalisation préfabriquées sont utilisées pour simuler les feux de signalisation dans notre maquette. Ces dispositifs reproduisent le comportement des feux de circulation réels, permettant de tester les algorithmes de gestion du trafic en conditions contrôlées. [10]

Avantages : Les LED de signalisation sont faciles à installer et à contrôler. Elles consomment peu d'énergie et offrent une grande durabilité, ce qui les rend idéales pour les systèmes de test. [11]

Inconvénients : La mise en place de ces LED nécessite un câblage précis et une configuration minutieuse pour simuler correctement les scénarios de trafic. [11]

Pourquoi ce choix : Les LED de signalisation préfabriquées ont été choisies pour leur facilité d'intégration et leur capacité à simuler de manière réaliste les cycles de feux de circulation, essentiels pour tester notre système de contrôle de trafic.

2.4. Carte RFID et Lecteur RFID (RC522)

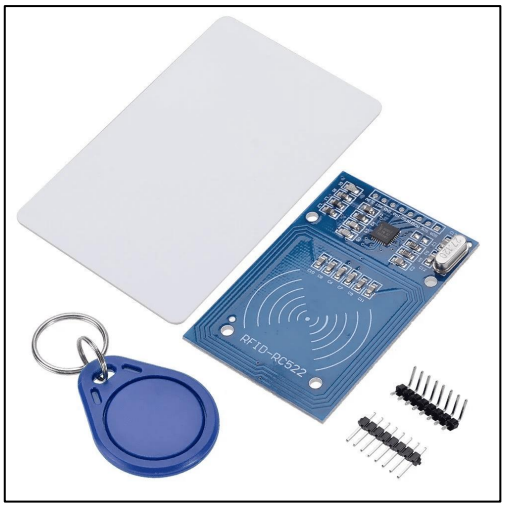


Figure 12: Carte RFID et Lecteur RFID

Informations Générales : Les cartes RFID et les lecteurs RFID sont utilisés pour identifier et suivre les véhicules équipés de ces cartes lorsqu'ils passent par des points de contrôle spécifiques. Cette technologie permet une identification rapide et sans contact des véhicules. [12]

Avantages : Les systèmes RFID offrent une identification rapide, fiable et précise des véhicules. Ils sont également robustes et fonctionnent bien dans diverses conditions environnementales. [12]

Inconvénients : Les lecteurs RFID ont une portée limitée, ce qui nécessite une installation soigneuse pour assurer une couverture adéquate. De plus, le coût des lecteurs et des cartes RFID peut s'accumuler rapidement dans des systèmes à grande échelle. [12]

Pourquoi ce choix : Nous avons choisi d'utiliser des cartes et lecteurs RFID pour leur capacité à fournir une identification précise des véhicules, ce qui est crucial pour la priorisation des véhicules d'urgence dans notre système de contrôle de trafic.

2.5. Heltec Wireless Stick



Figure 13: Heltec Wireless Stick

Informations Générales : Le Heltec Wireless Stick est un module de communication intégrant à la fois le Wi-Fi et le LoRa, offrant une solution polyvalente pour les applications IoT nécessitant différentes méthodes de communication. [13]

Avantages : Le Heltec Wireless Stick est compact, polyvalent et facile à programmer. Il permet une communication fiable et est supporté par une large communauté de développeurs. [13]

Inconvénients : La configuration et la programmation du Heltec Wireless Stick peuvent nécessiter des compétences techniques spécifiques. De plus, la gestion de l'alimentation et des interférences peut être un défi. [13]

Pourquoi ce choix : Nous avons choisi le Heltec Wireless Stick pour sa polyvalence et sa capacité à intégrer à la fois le Wi-Fi et le LoRa, permettant une communication flexible et fiable dans notre système.

2.6. Câbles Jumper



Figure 14: Câbles Jumper

Informations Générales : Les câbles jumper sont utilisés pour établir des connexions temporaires entre les composants électroniques sur des plaques de prototypage (breadboards). Ils sont indispensables pour le prototypage et les tests de circuits électroniques. [12]

Avantages : Les câbles jumper sont flexibles, faciles à utiliser et permettent des connexions rapides et réversibles entre les composants. Ils sont essentiels pour tester et développer des prototypes électroniques. [12]

Inconvénients : L'utilisation excessive de câbles jumper peut rendre les circuits encombrants et compliquer la gestion des connexions, surtout dans les projets de grande envergure. [12]

Pourquoi ce choix : Les câbles jumper ont été choisis pour leur simplicité et leur flexibilité, permettant de créer rapidement des connexions temporaires nécessaires pour le prototypage et les tests de notre système de contrôle de trafic.

3. Choix des technologies

3.1. LoRa



Figure 15: LoRa

Informations Générales : LoRa (Long Range) est une technologie de communication sans fil utilisée pour les réseaux longue portée à faible consommation d'énergie. Elle est particulièrement adaptée aux applications IoT nécessitant des communications à longue distance. [14]

Avantages : LoRa offre une portée de communication étendue et une faible consommation d'énergie, ce qui est idéal pour les dispositifs distants. Elle permet également une bonne pénétration dans les environnements urbains et les bâtiments. [14]

Inconvénients : Le débit de données de LoRa est limité, ce qui peut restreindre son utilisation pour des applications nécessitant une bande passante élevée. [14]

Pourquoi ce choix : Nous avons choisi LoRa pour sa capacité à assurer une communication fiable sur de longues distances avec une faible consommation d'énergie, adaptée aux besoins de notre système de gestion du trafic.

3.2. Protocole MQTT



Figure 16: MQTT

Informations Générales : MQTT (Message Queuing Telemetry Transport) est un protocole de messagerie léger, conçu pour les applications IoT et M2M, facilitant la communication fiable et asynchrone entre appareils connectés. [15]

Avantages :

- Modèle Pub/Sub : Communication asynchrone.
- Légèreté : Réduit la surcharge réseau.
- Fiabilité : Niveaux de QoS pour la livraison des messages.
- Évolutivité : Adapté aux architectures distribuées.
- Sécurité : Authentification, autorisation et chiffrement. [15]

Inconvénients :

- Complexité de Configuration : Nécessite des compétences techniques.
- Gestion des Ressources : Peut encore être exigeant en environnement très contraint. [15]

Pourquoi ce choix : Nous avons choisi MQTT pour sa légèreté, son efficacité et sa robustesse, essentielles pour la communication rapide et fiable dans notre système de contrôle de trafic intelligent.

3.3. HiveMQ



Figure 17: HiveMQ

Informations Générales : HiveMQ est un broker MQTT performant et scalable, conçu pour les applications IoT. Il offre des fonctionnalités avancées pour la gestion des communications MQTT à grande échelle. [16]

Avantages : HiveMQ supporte de nombreux clients simultanés, offre des performances élevées et une grande fiabilité. Il dispose également de fonctionnalités de sécurité avancées pour protéger les communications. [16]

Inconvénients : La complexité de HiveMQ peut poser des défis pour les débutants, et sa configuration nécessite des compétences techniques spécifiques. [16]

Pourquoi ce choix : Nous avons choisi HiveMQ pour ses capacités de gestion des communications à grande échelle et ses fonctionnalités de sécurité, essentielles pour notre projet de contrôle de trafic intelligent.

3.4. Modèle YOLOv8n



Figure 18: YOLOv8

Informations Générales : Le modèle YOLOv8n (You Only Look Once version 8n) est un algorithme de vision par ordinateur de pointe pour la détection d'objets en temps réel. Il est connu pour sa rapidité et sa précision dans l'identification et la localisation des objets dans les images. [17]

Avantages : YOLOv8n offre une haute précision et une rapidité de traitement exceptionnelle, ce qui le rend idéal pour les applications nécessitant une détection d'objets en temps réel. Il est également bien supporté par la communauté de recherche en vision par ordinateur. [17]

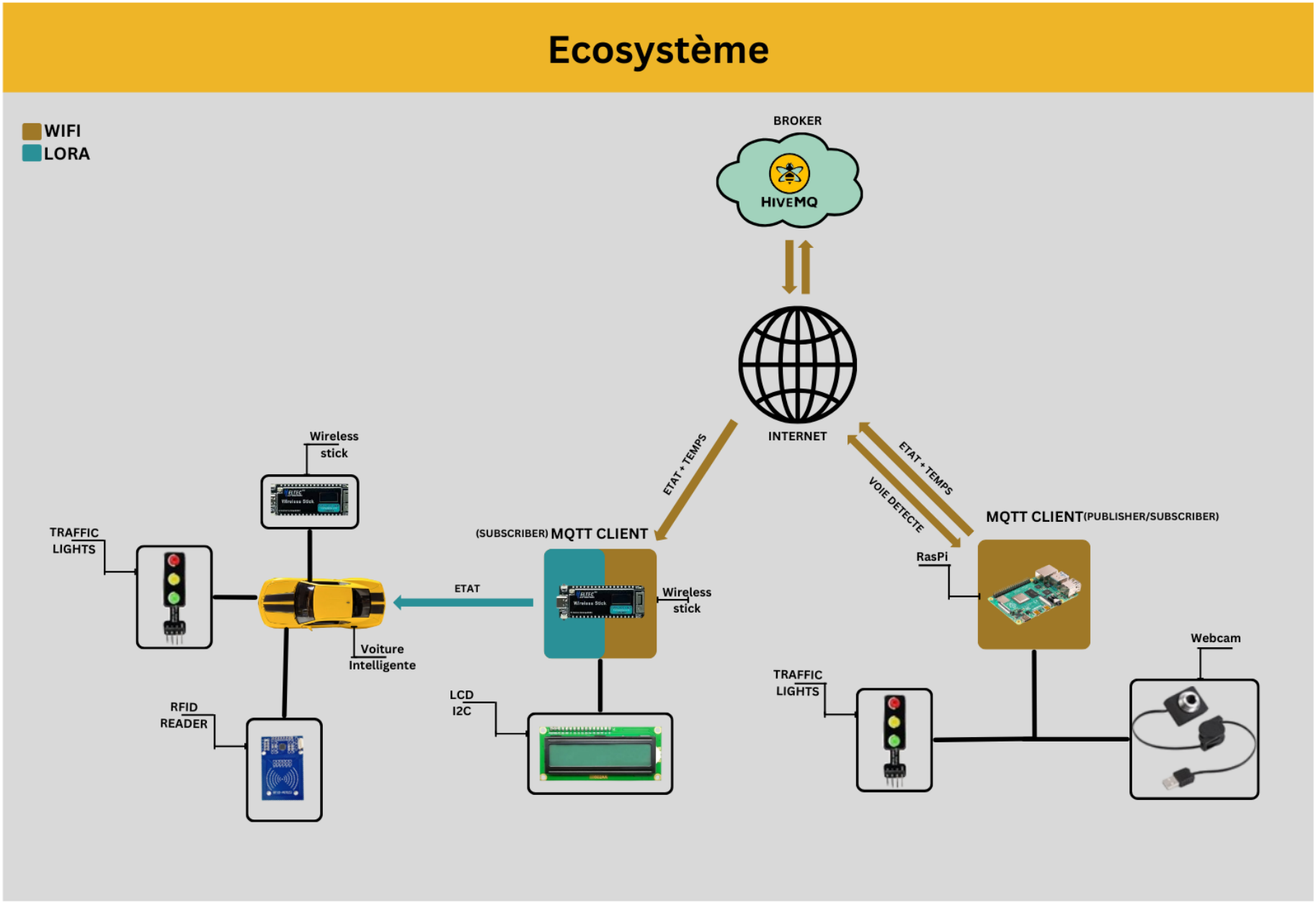
Inconvénients : L'entraînement du modèle YOLOv8n nécessite des ressources informatiques importantes et une grande quantité de données étiquetées. De plus, l'optimisation du modèle pour des cas d'utilisation spécifiques peut être complexe. [17]

Pourquoi ce choix : Nous avons choisi le modèle YOLOv8n pour ses performances supérieures en détection d'objets, essentielles pour identifier rapidement et précisément les véhicules d'urgence dans notre système de contrôle de trafic.

4. Ecosystème

Nous avons choisi de diviser notre explication du schéma en deux parties distinctes afin de mieux mettre en évidence les différentes fonctionnalités et composants de notre système complexe : la première partie se concentre sur la gestion des véhicules prioritaires, tandis que la seconde partie aborde la notification des véhicules intelligents, permettant ainsi une compréhension claire et organisée de notre écosystème.

4.1. Schéma


Ecosystème
WIFI
LORA
BROKER
HIVEMQ
INTERNET
ETAT + TEMPS
ETAT + TEMPS
VOIE DETECTE
MQTT CLIENT(PUBLISHER/SUBSCRIBER)
RasPi
(SUBSCRIBER) MQTT CLIENT
ETAT
Wireless stick
Wireless stick
TRAFFIC LIGHTS
Voiture Intelligente
RFID READER
LCD I2C
TRAFFIC LIGHTS
Webcam

4.2. Partie gestion des voitures prioritaires

Cette partie du système est dédiée à la gestion des véhicules d'urgence. Lorsqu'un véhicule d'urgence est détecté par la « Webcam », le « RasPi » publie le chemin emprunté par ce véhicule sur le broker HiveMQ sous le topic "SMdetection". Simultanément, le « RasPi » s'abonne à ce même topic et, en fonction de la valeur reçue, il modifie les feux de signalisation installés au carrefour « TRAFFIC LIGHTS » en conséquence.

4.3. Partie notification des véhicules intelligents

Cette section implique l'utilisation de deux cartes « Wireless Stick » : l'une est un émetteur LoRa et l'autre un récepteur LoRa.

Chaque seconde, le « RasPi » publie l'état et le temps restant avant le changement des feux de signalisation sur le broker sous le topic "SMTraffic". L'émetteur LoRa « Wireless Stick », abonné à ce topic, imprime en temps réel les valeurs reçues sur un écran « LCD I2C » et les transmet également via LoRa.

Le récepteur LoRa « Wireless Stick », installé dans ce que nous appelons la « Voiture Intelligente », reçoit les données transmises par l'émetteur LoRa « Wireless Stick ». Lorsqu'une carte RFID de voie est détectée par le lecteur RFID, les feux de signalisation « TRAFFIC LIGHTS » de la « Voiture Intelligente » changent en fonction des valeurs reçues en temps réel via LoRa. Ces valeurs correspondent à celles de la voie spécifique car chaque voie a ses propres feux de signalisation.

Lorsque la carte RFID d'un carrefour est détectée par le lecteur RFID « RFID READER », les feux de signalisation « TRAFFIC LIGHTS » de la « Voiture Intelligente » s'éteignent, indiquant qu'il n'est plus nécessaire de continuer à afficher l'état de cette voie après que la « Voiture Intelligente » l'a déjà traversée.

5. Schémas et montages

5.1. Connexion du Raspberry Pi 4

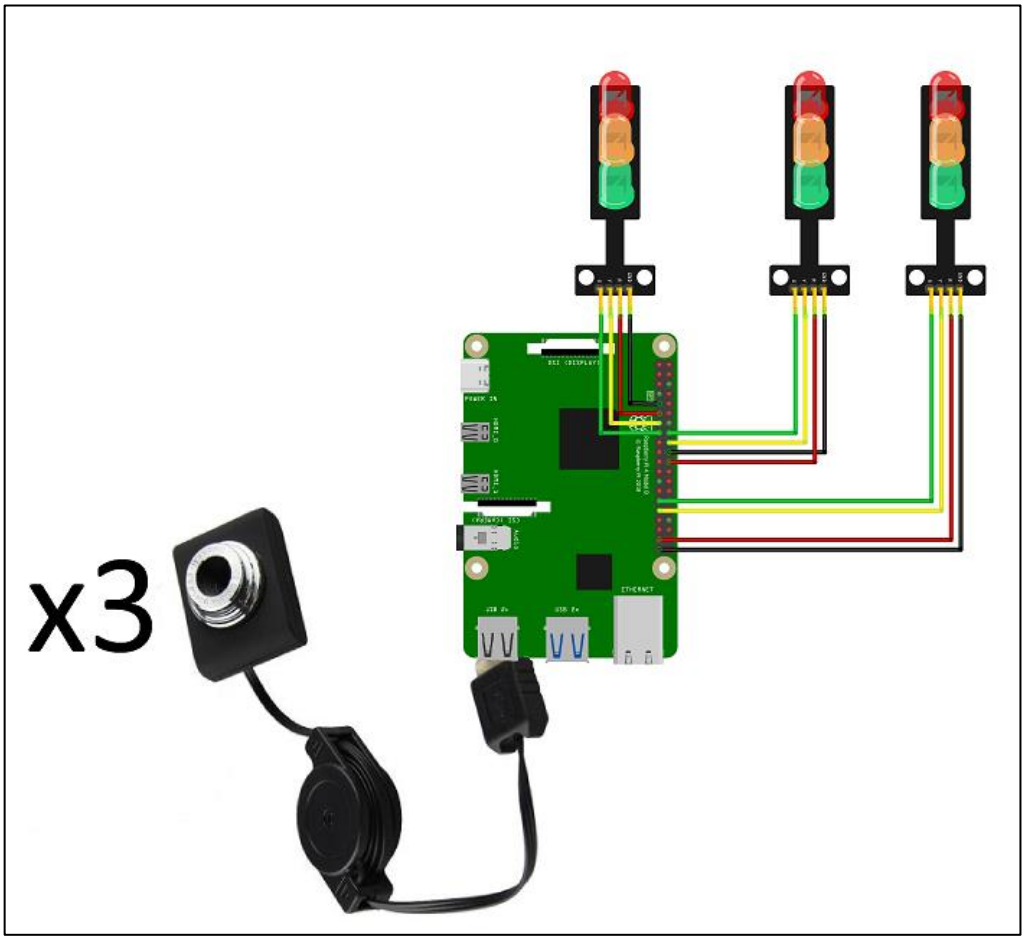


Figure 19: Connexion du Raspberry Pi 4

- GPIO22 => vert_voie 1
- GPIO17 => rouge_voie 1
- GPIO27 => jaune_voie 1
- GPIO23 => vert_voie 2
- GPIO25 => rouge_voie 2
- GPIO24 => jaune_voie 2
- GPIO5 => vert_voie 3
- GPIO26 => rouge_voie 3
- GPIO6 => jaune_voie 3

5.3. Connexion du Heltec Wireless Stick (récepteur LoRa)

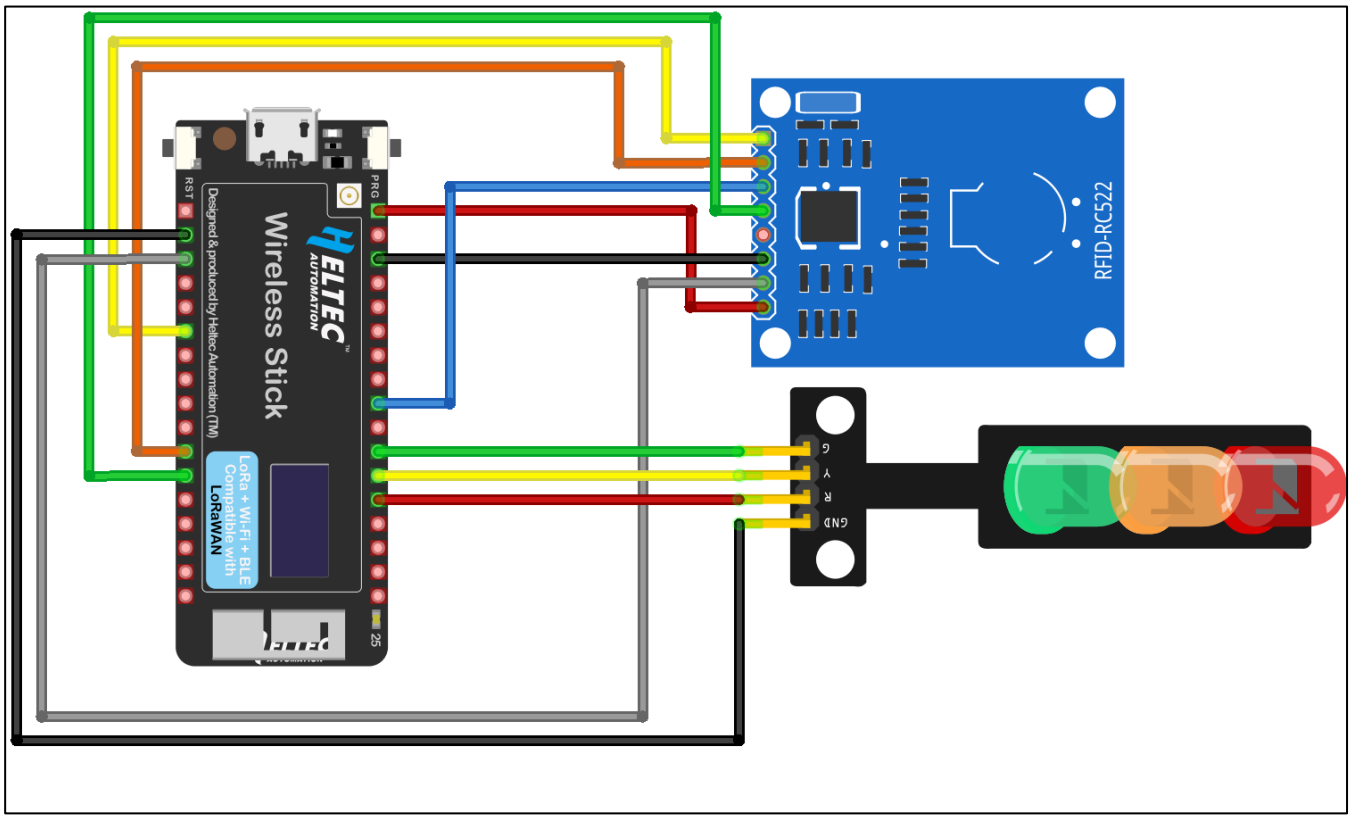


Figure 20: Connexion du Heltec Wireless Stick (récepteur LoRa)

- GPIO12 => rouge
- GPIO13 => jaune
- GPIO33 => vert
- GPIO21 => SDA
- GPIO5 => SCK
- GPIO27 => MOSI
- GPIO19 => MISO
- GPIO22 => RST

5.2. Connexion du Heltec Wireless Stick (émetteur LoRa)

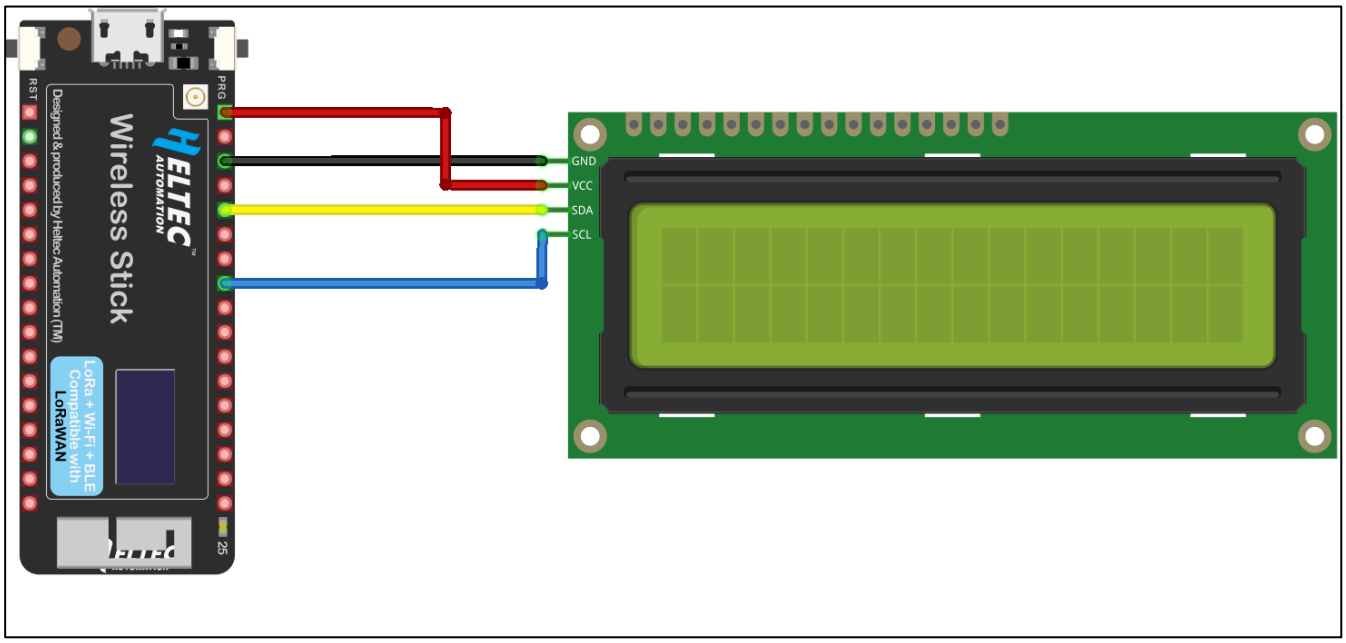


Figure 21: Connexion du Heltec Wireless Stick (émetteur LoRa)

- GPIO4 => SDA
- GPIO15 => SCL

Conclusion

En conclusion, ce chapitre a méticuleusement décrit la conception et les composants de notre système de contrôle de trafic intelligent. En explorant les différentes intersections et technologies impliquées, nous avons mis en évidence les considérations et décisions qui sous-tendent notre système. Cet examen détaillé démontre non seulement la sophistication et la complexité de la solution de gestion du trafic, mais souligne également son potentiel à améliorer l'efficacité et la sécurité du trafic. Grâce à un placement stratégique des caméras et à des algorithmes avancés, notre système est prêt à gérer efficacement les flux de trafic urbains, assurant une expérience de transit fluide et sécurisée pour tous les usagers de la route.

Chapitre 5 : Réalisation

Introduction

Dans ce chapitre, nous abordons la mise en œuvre pratique de notre système intelligent de gestion du trafic. Cette section est dédiée à la description détaillée des étapes nécessaires à l'installation et à la configuration des différents composants matériels et logiciels de notre solution. Nous commencerons par l'installation du système d'exploitation Raspbian sur un Raspberry Pi, puis nous aborderons la configuration des caméras, l'installation et la configuration des différents logiciels nécessaires, ainsi que l'entraînement du modèle YOLOv8 pour la détection des objets. Enfin, nous détaillerons la construction de la maquette de l'intersection et de la voiture intelligente. L'objectif est de fournir une vue d'ensemble complète de la réalisation de notre projet, en mettant l'accent sur les aspects techniques et les défis rencontrés.

1. Installation de Raspbian sur Raspberry Pi

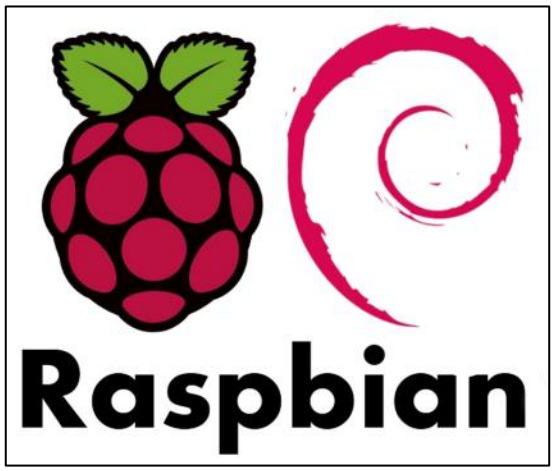


Figure 22: Raspbian

1.1. Téléchargement et Installation de SD Card Formatter

Téléchargement de SD Card Formatter à partir du site officiel :



Figure 23: Téléchargement de SD Card Formatter

Installation du logiciel sur votre ordinateur :

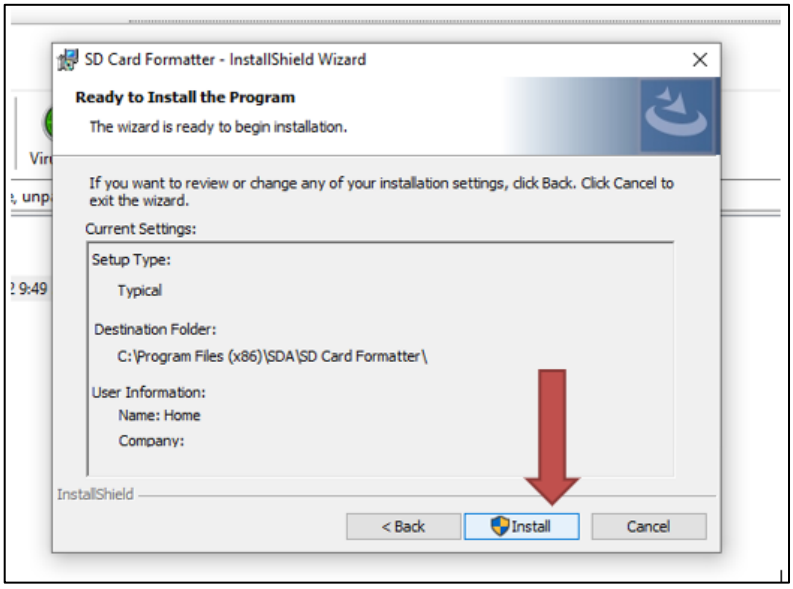


Figure 24: Installation du SD Card Formatter

1.2. Formatage de la Carte SD

Insertion la carte SD dans le lecteur de carte de votre ordinateur.

Ouverture de SD Card Formatter.

Sélection de la carte SD et cliquer sur "Format" pour effacer complètement son contenu :

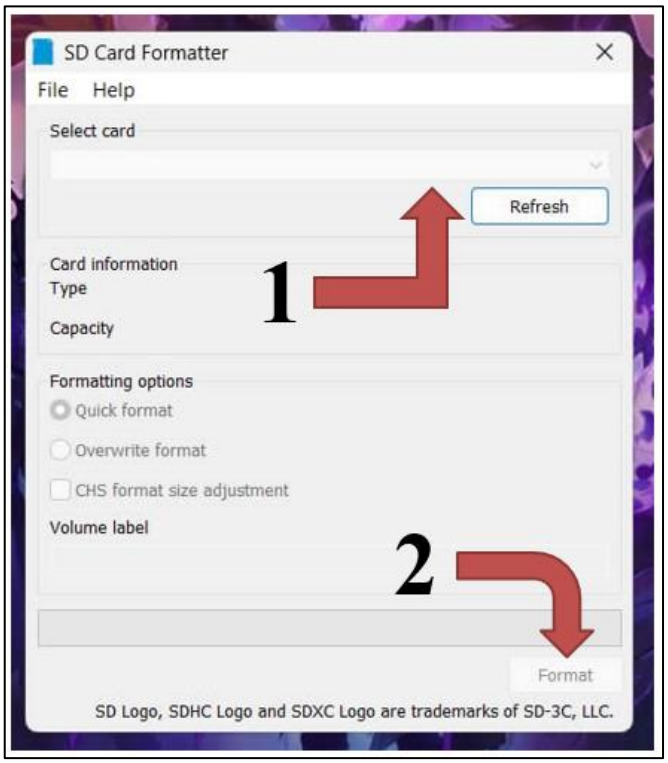


Figure 25: Formatage de la Carte SD

1.3. Téléchargement de Berryboot

Téléchargement de Berryboot depuis son site officiel :



Figure 26: Téléchargement de Berryboot

1.4. Copie de Berryboot sur la Carte SD

Extraction du contenu du fichier téléchargé :

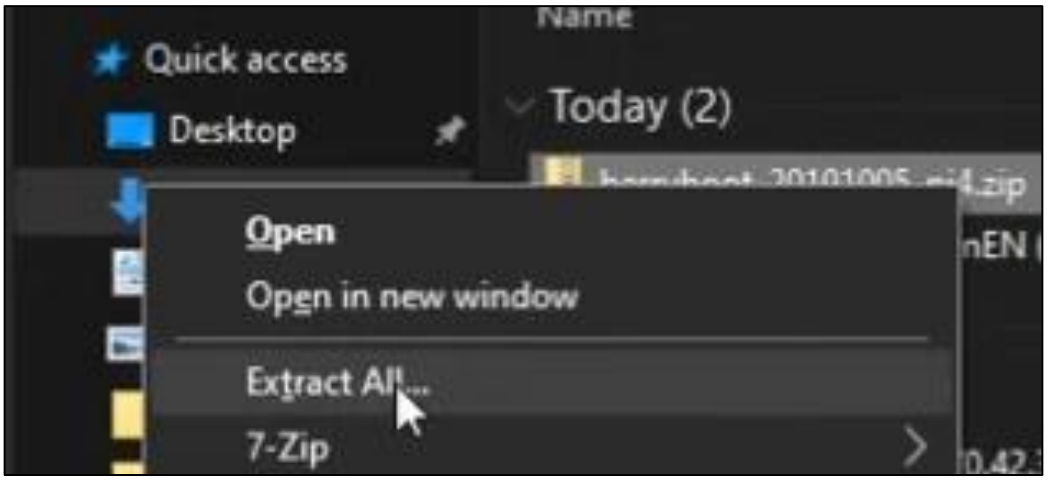


Figure 27: Extraction de contenu du fichier téléchargé

Copie des fichiers extraits sur la carte SD précédemment formatée.

1.5. Insertion de la Carte SD dans le Raspberry Pi

Insertion de la carte SD contenant Berryboot dans le Raspberry Pi.

1.6. Initialisation de Berryboot

Allumage du Raspberry Pi :



Figure 28: Etape 1 d'initialisation de Berryboot



Figure 29: Etape 2 d'initialisation de Berryboot



Figure 30: Etape 3 d'initialisation de Berryboot

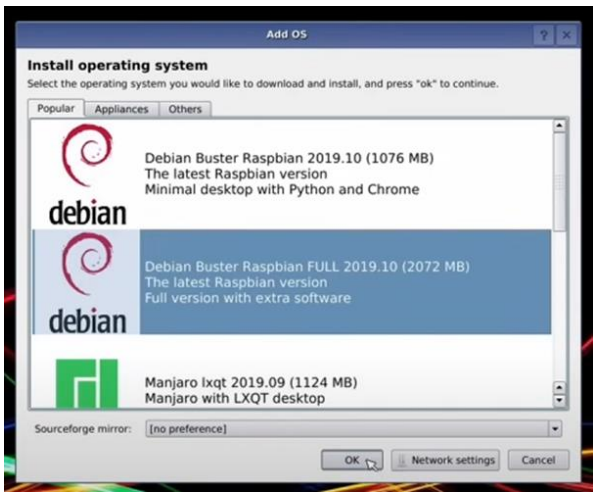


Figure 31: Etape 4 d'initialisation de Berryboot

2. Configuration de VNC sur Raspberry Pi

2.1. Installation de RealVNC sur Windows



Figure 32: Installation de RealVNC

2.2. Activation du serveur VNC

Utilisation du sudo raspi-config, puis aller dans "Interfacing Options" et activer VNC : [18]

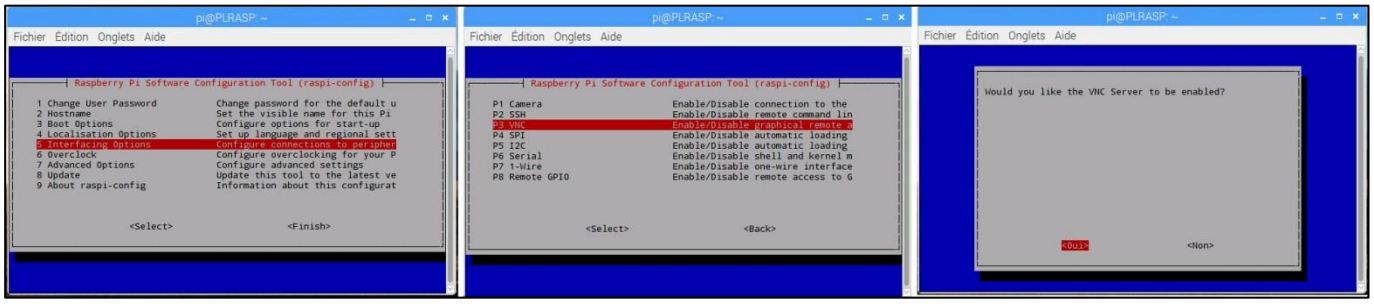


Figure 33: Activation du serveur VNC

2.3. Configuration du réseau

Connexion au Wi-Fi.

Configuration de l'adresse IP du Raspberry Pi en une adresse IP statique : [18]

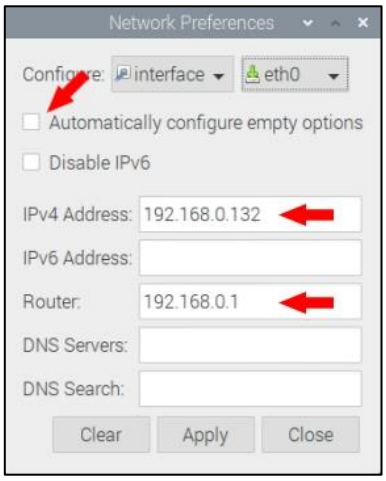


Figure 34: Configuration d'adresse IP (statique)

2.4. Connexion depuis un autre appareil

Utilisation de VNC Viewer sur un PC ou mobile pour se connecter à l'adresse IP du Raspberry Pi :

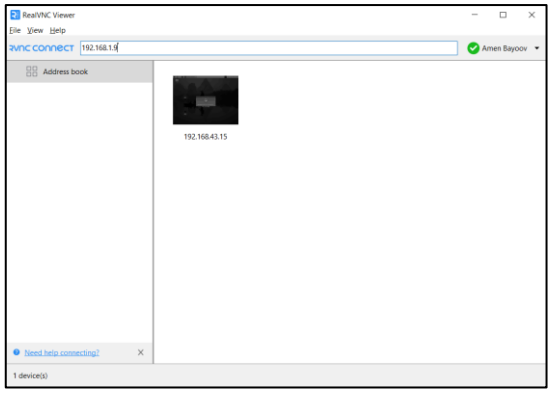


Figure 35: Insertion d'adresse IP du Raspberry sur le VNC

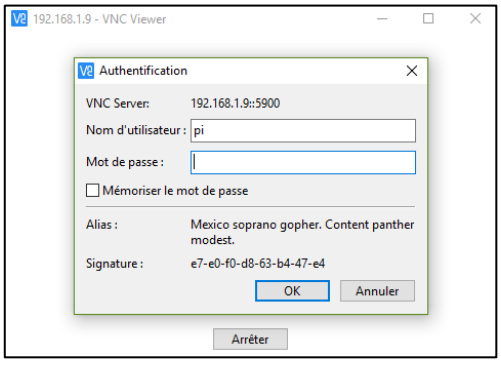


Figure 36 : Authentification du VNC

3. Configuration des Caméras

3.1. Connexion des caméras

Connexion des caméras USB aux ports USB du Raspberry Pi.

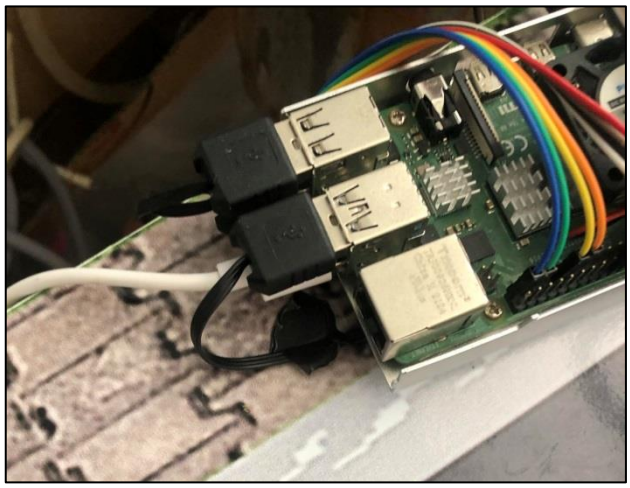


Figure 37: Connexion des caméras

3.2. Installation des bibliothèques nécessaires

Installation d'OpenCV avec la commande pip install opencv-python :

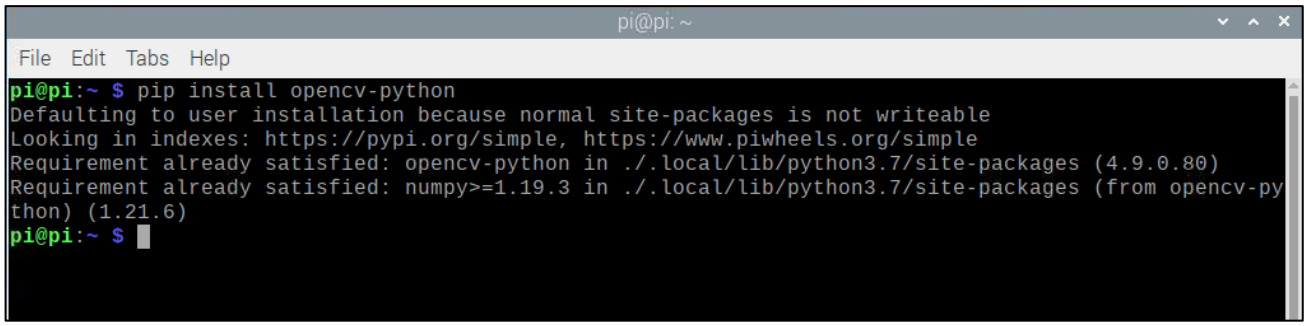


Figure 38: Installation d'OpenCV

3.3. Identification des indices des caméras

Utilisation d'un script Python pour identifier les indices des caméras connectées.[19]

4. Étiquetage des Images pour l'Entraînement du Modèle YOLOv8

4.1. Installation de LabelImg

Installation de LabelImg avec la commande pip install labelImg :

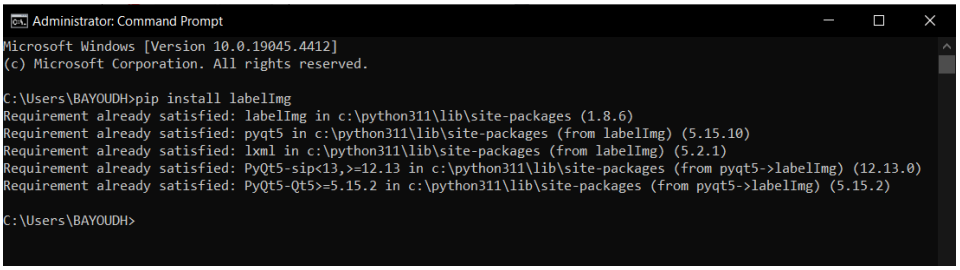


Figure 39: Installation de LabelImg

4.2. Ouverture de LabelImg

Lancement de LabelImg et ouverture du dossier contenant les images :

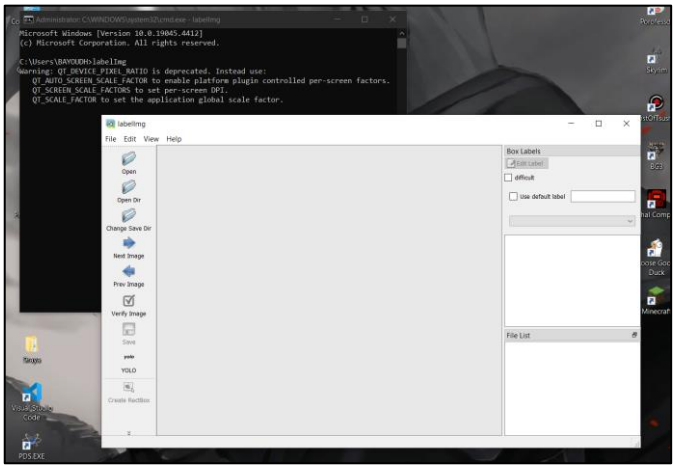


Figure 40: Ouverture de LabelImg

4.3. Étiquetage des images

Utilisation de LabelImg pour étiqueter les véhicules prioritaires dans chaque image : [20]

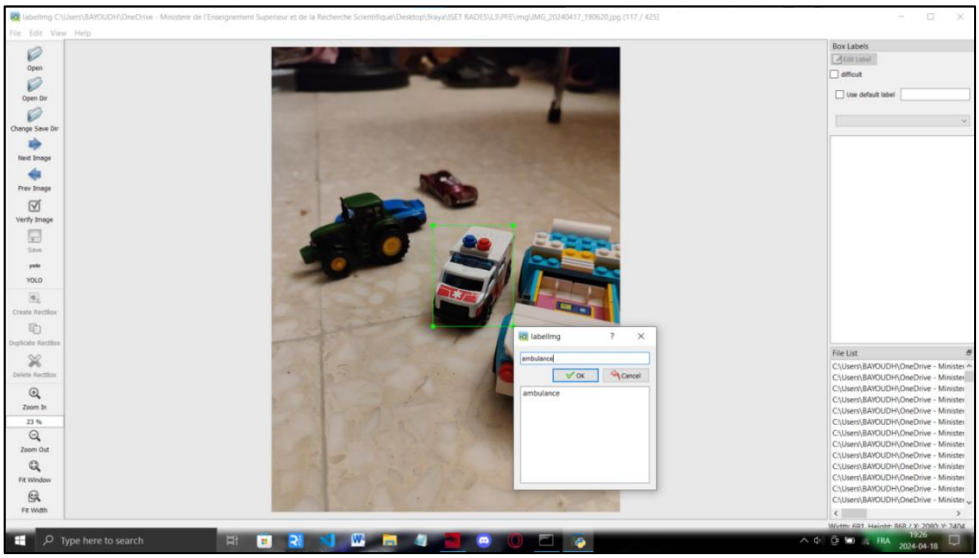


Figure 41: Étiquetage des images

4.4. Sauvegarde des annotations

Sauvegarde des annotations dans le format requis (par exemple, XML ou TXT).

5. Entraînement du Modèle YOLOv8 sur VS Code

5.1. Préparation de l'environnement

Configuration de VS Code avec l'extension Python :



Figure 42: Configuration de VS Code avec l'extension Python

5.2. Chargement des données d'entraînement

Chargement des images et des fichiers d'annotations dans l'environnement de travail.

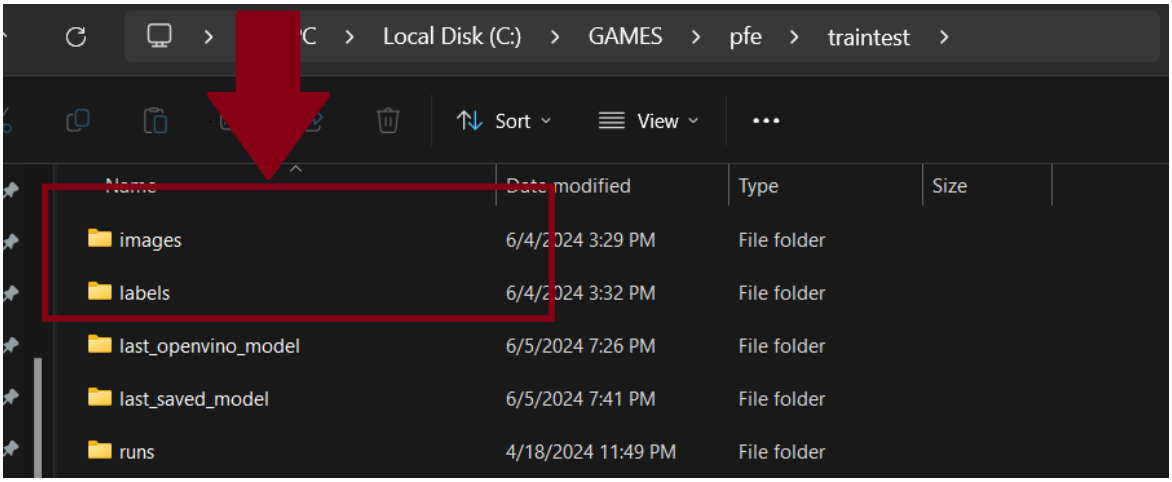


Figure 43: Chargement des données d'entraînement

5.3. Script d'entraînement

Écriture et lancement du script d'entraînement pour YOLOv8 en utilisant les données étiquetées.

5.4. Visualisation de l'entraînement

Surveillance de la progression de l'entraînement et visualisation des graphiques de perte et de précision.

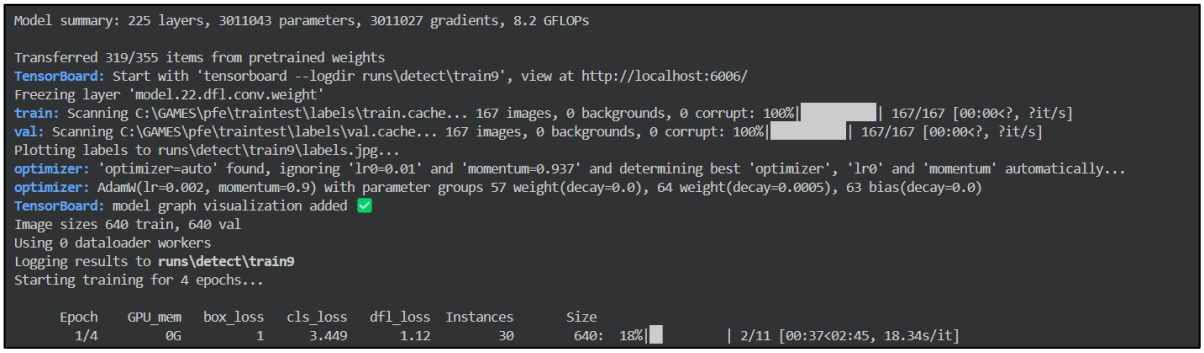


Figure 44: Visualisation de l'entraînement

5.5. Validation du modèle

Validation du modèle entraîné sur un ensemble de données de test et visualisation des résultats.

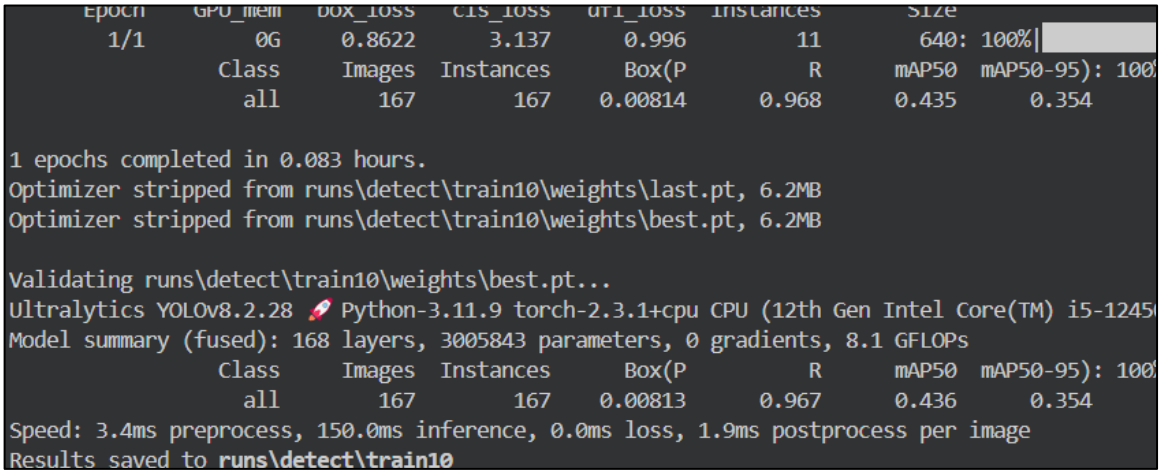


Figure 45: Validation du modèle

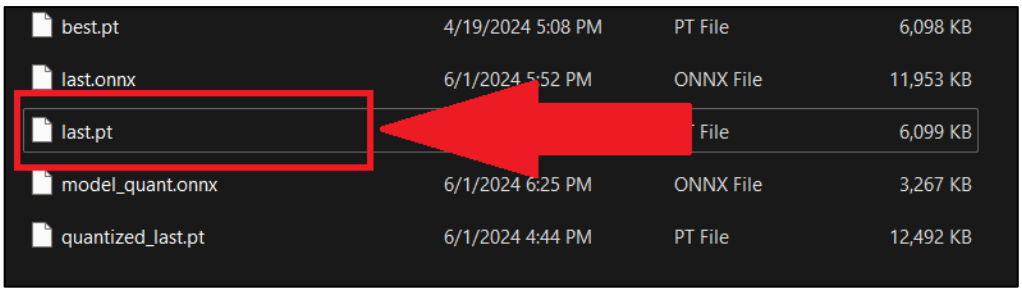


Figure 46: Modèle entraîné

5.6. Analyse des résultats

Perte et Métriques d'Entraînement et de Validation :

Les graphiques d'entraînement et de validation fournissent des informations sur le processus d'apprentissage de notre modèle YOLOv8n. Sur 100 époques, les métriques de perte (perte de boîte, perte de classe et perte dfl) ont constamment diminué, démontrant l'amélioration de la précision du modèle. Les métriques de précision et de rappel, ainsi que les scores mAP, sont restés élevés, montrant la forte performance du modèle pour identifier correctement les ambulances et sa précision générale dans la gestion de l'ensemble de données.

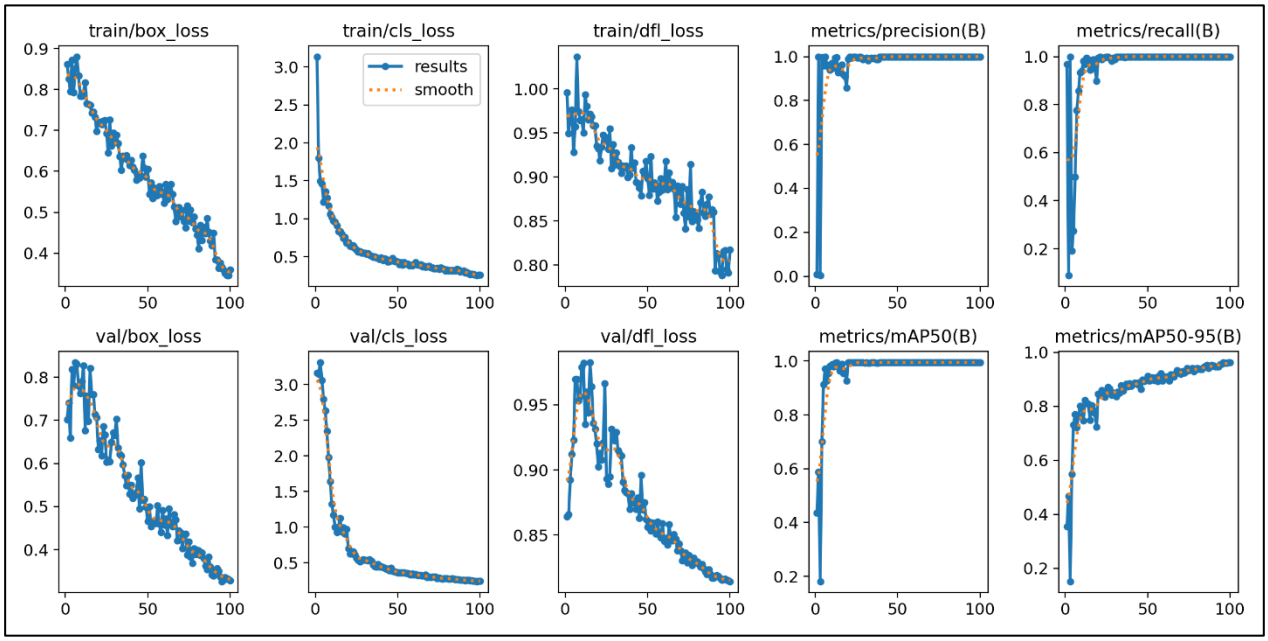


Figure 47: Résultat du modèle

Matrice de Confusion :

La matrice de confusion dans notre projet montre la performance de notre modèle YOLOv8n entraîné sur notre ensemble de données personnalisé d'images d'ambulance. Dans cette matrice, le modèle a correctement identifié toutes les 167 images d'ambulance sans aucune erreur, indiquant qu'il est très efficace pour distinguer les ambulances de l'arrière-plan.

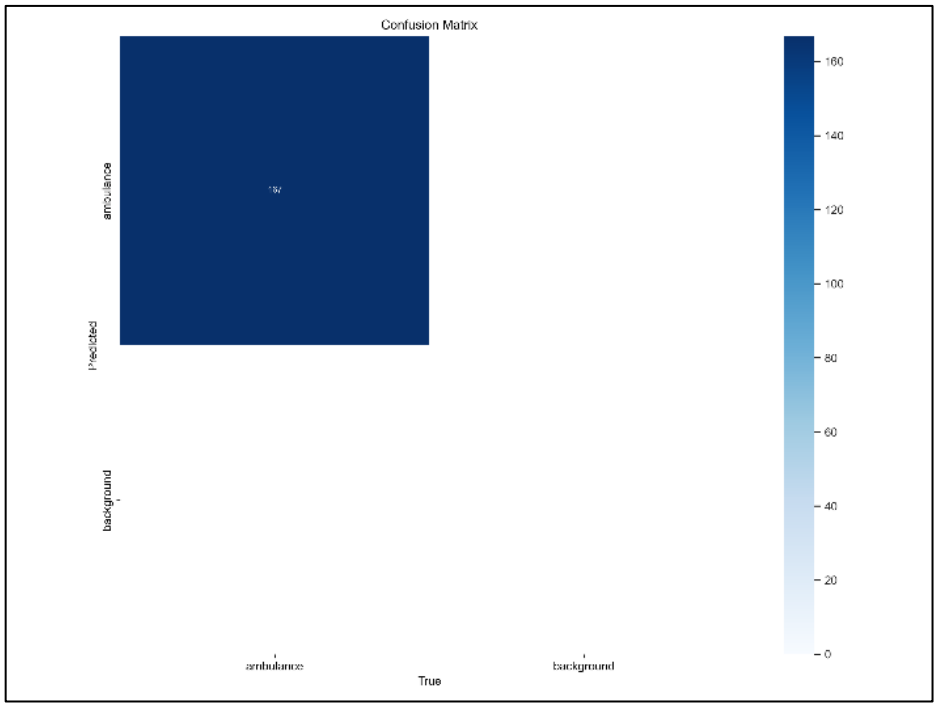


Figure 48: Confusion Matrix

Courbe de Confiance F1 :

La courbe de confiance F1 illustre la relation entre le score F1 de notre modèle et le niveau de confiance avec lequel il fait des prédictions. La courbe montre que notre modèle atteint un score F1 proche de 1.0, en particulier lorsque le seuil de confiance est autour de 0.824. Cela indique que le modèle est très précis et fiable dans ses prédictions.

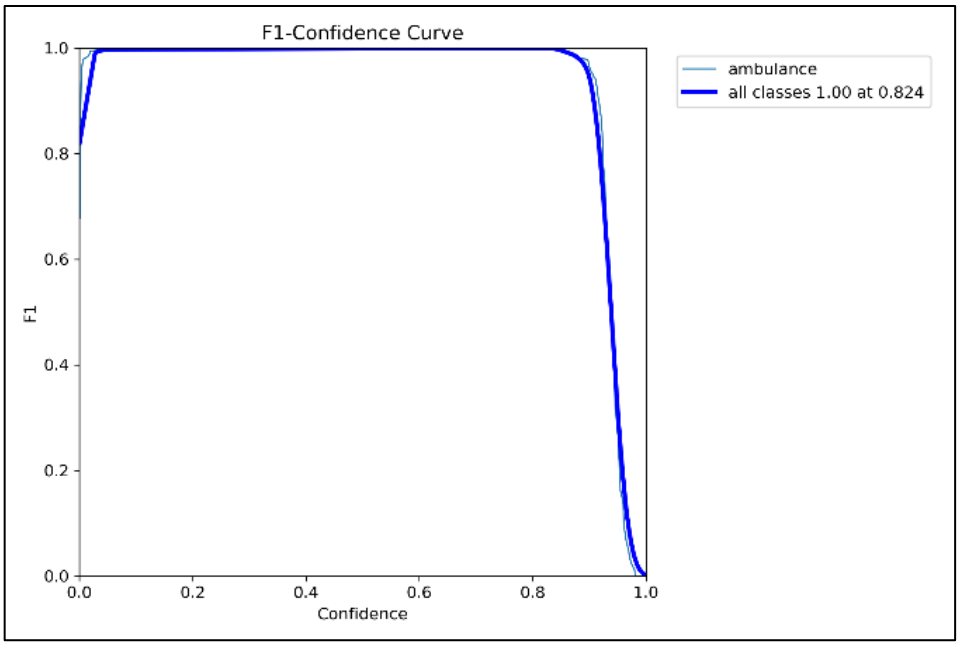


Figure 49: F1-Confidence Curve

6. Construction du Maquette

6.1. Réalisation de l'image de l'intersection et impression sur contreplaqué

Box de Commande :
Ras Pi + Heltec (LoRa Sender)



Figure 50: Maquette

Emplacement des cartes RFID



Figure 51: Maquette vue de dessous

6.2. Pose des poteaux de feux au carrefour

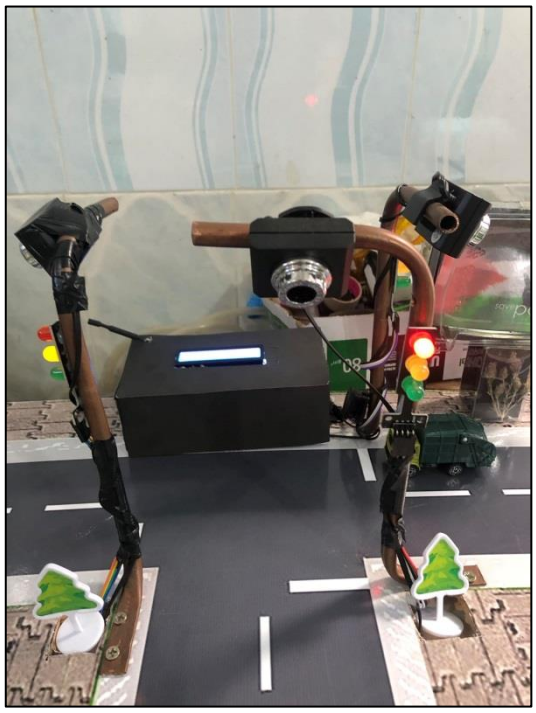


Figure 52: Emplacement des poteaux

6.3. Placement des caméras et des feux de signalisation préfabriqués dans chaque voie



Figure 53: Placement des caméras et des feux

7. Construction de la voiture intelligente

7.1. Solution Choisie : Intégration du système LoRa avec RFID

Après une analyse approfondie des différentes solutions potentielles, nous avons opté pour une approche combinant la technologie LoRa avec la détection RFID pour notre prototype de gestion du trafic. Voici un aperçu de cette solution :

Implémentation de la technologie LoRa :

- Nous avons choisi d'utiliser la technologie LoRa en raison de sa faible consommation d'énergie et de sa capacité à transmettre des données sur de longues distances.
- Un HeltecWirelessStick a été placé au centre de l'intersection, agissant comme le point central de communication.
- Un autre HeltecWirelessStick a été installé dans le véhicule intelligent, connecté à un détecteur RFID intégré.
- Le HeltecWirelessStick central communique avec le véhicule via LoRa pour fournir des informations sur l'état des feux de signalisation et adapter la conduite en conséquence.

Intégration de la détection RFID :

- Des étiquettes RFID ont été placées sous la route le long de chaque voie pour fournir des identifiants uniques aux véhicules.
- Un lecteur RFID a été installé dans le véhicule intelligent pour détecter les étiquettes RFID sous la route.
- En fonction des étiquettes RFID détectées, le véhicule peut déterminer sa position par rapport à l'intersection et adapter sa conduite en conséquence.

Cette solution combine l'efficacité énergétique de LoRa avec la précision de la détection RFID, offrant ainsi une communication fiable et une gestion efficace du trafic pour améliorer la fluidité et la sécurité des déplacements dans notre prototype.[21]

7.2. Prototype de la voiture de test

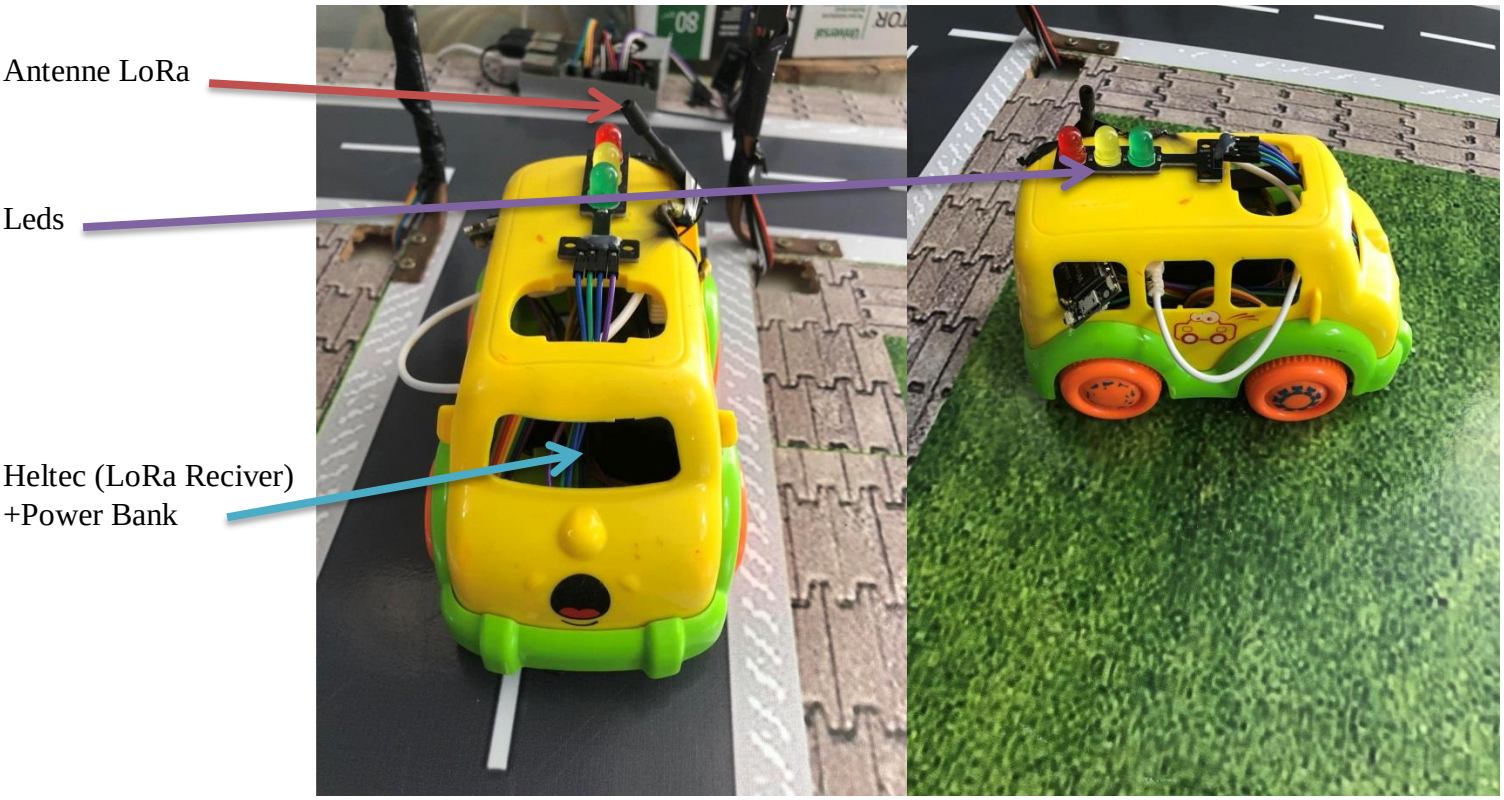


Figure 54: Prototype de la voiture de test



Figure 55: Vue de dessous de la voiture de test

8. Défis Majeurs Rencontrés et Solutions

8.1. Délai de Détection avec la Raspberry Pi

L'un des défis les plus agaçants que nous avons rencontrés concernait le délai de détection lors de l'utilisation de la caméra connectée à la Raspberry Pi 4 pour tester la vitesse de détection avec le modèle YOLOv8n. Nous avons observé un délai significatif, particulièrement visible lorsque nous avons tenté de faire fonctionner simultanément les trois caméras. En effet, lorsque l'ensemble du programme tournait sur la Raspberry Pi seule, qui n'est pas aussi puissante qu'un PC, le délai de détection atteignait entre 24 et 27 secondes, ce qui était inacceptable.

Solution Apportée :

Pour atténuer ce problème, nous avons pris plusieurs mesures :

- Nous avons modifié le code pour ignorer quelques images si nécessaire, afin d'améliorer les performances.
- Nous avons réduit la résolution des caméras.
- Nous avons utilisé le multiprocessing au lieu du threading pour gérer les trois caméras. Cela permettait d'assurer à la fois précision et performance, bien que cette méthode consomme plus de ressources que le threading.
- Nous avons également abandonné le code qui comptait les voitures dans chaque voie avant de passer au feu rouge, ce qui allégeait considérablement le traitement sur la Raspberry Pi. Cette fonctionnalité initiale permettait d'ajuster la durée des feux en fonction du nombre de véhicules présents, mais en la supprimant, nous avons libéré des ressources pour nous concentrer sur la détection des véhicules prioritaires et améliorer la réactivité des feux de signalisation.

Grâce à ces ajustements, nous avons réussi à réduire le temps de détection à une plage de 4 à 8 secondes. Nous avons également testé d'autres modèles de détection comme YOLOv5s, YOLOv5n et YOLOv3-tiny, mais aucune amélioration notable n'a été observée, confirmant que la Raspberry Pi était trop limitée pour ce type de traitement intensif.

8.2. Gestion de la Connexion entre les Codes

Un autre défi majeur était la gestion de la connexion entre les différents codes sur la même Raspberry Pi. En effet, nous devions lancer les trois caméras pour la détection tout en assurant le changement cohérent des feux de signalisation dans chaque voie de manière continue. De plus, il était crucial d'arrêter

le changement d'état des feux lorsqu'une ambulance était détectée et d'implémenter un état personnalisé pour les feux pendant un certain temps avant de revenir au flux normal.

Solution Apportée :

Initialement, nous avons tenté d'envoyer un signal via « `os.kill` » pour déclencher une fonction séparée qui gérerait cette situation. Cependant, cette approche s'est avérée chaotique et inefficace. Nous avons donc adopté une approche différente :

- Nous avons configuré la Raspberry Pi pour qu'elle fonctionne comme son propre éditeur (publisher) et abonné (subscriber), facilitant ainsi la communication entre ses différents codes séparés.
- En tirant parti du lancement soudain de la fonction de rappel (callback), nous avons pu effectuer les changements nécessaires de manière ordonnée.

Cette solution a permis d'éliminer le problème, assurant une gestion fluide et cohérente de l'état des feux de signalisation en fonction des détections des caméras. [22]

8.3. Problèmes avec l'Intégration de HeltecWirelessStick et du Lecteur RFID

Lorsque nous avons utilisé la carte HeltecWirelessStick avec le lecteur RFID, chaque composant fonctionnait bien séparément. Cependant, lorsque nous les avons intégrés pour qu'ils reçoivent des données via LoRa et lisent les cartes RFID simultanément, nous avons rencontré des problèmes. Après un débogage minutieux de notre code, nous avons identifié que l'initialisation de la carte HeltecWirelessStick avec "heltec.begin" interférait avec les broches SPI.

Solution Apportée :

Pour résoudre ce problème, nous avons modifié la bibliothèque utilisée pour configurer LoRa. Au lieu d'utiliser la bibliothèque "heltec.h", nous avons opté pour "LoRa.h", ce qui nous a permis de configurer manuellement les broches SPI. Bien que nous ayons dû renoncer à l'affichage du temps restant, nous avons pu atteindre notre objectif principal de faire fonctionner la communication prioritaire, ce qui en valait la peine. [21]

9. Test Final

9.1. Cas neutre

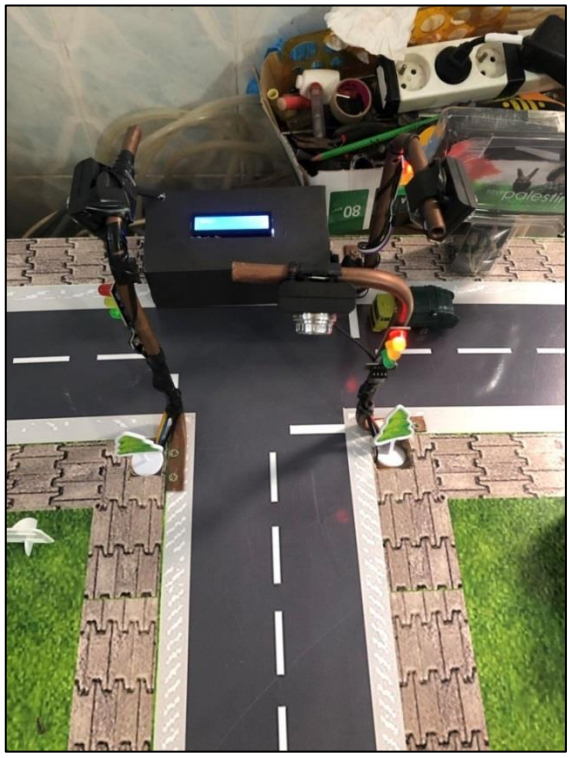


Figure 56: État normal du feu

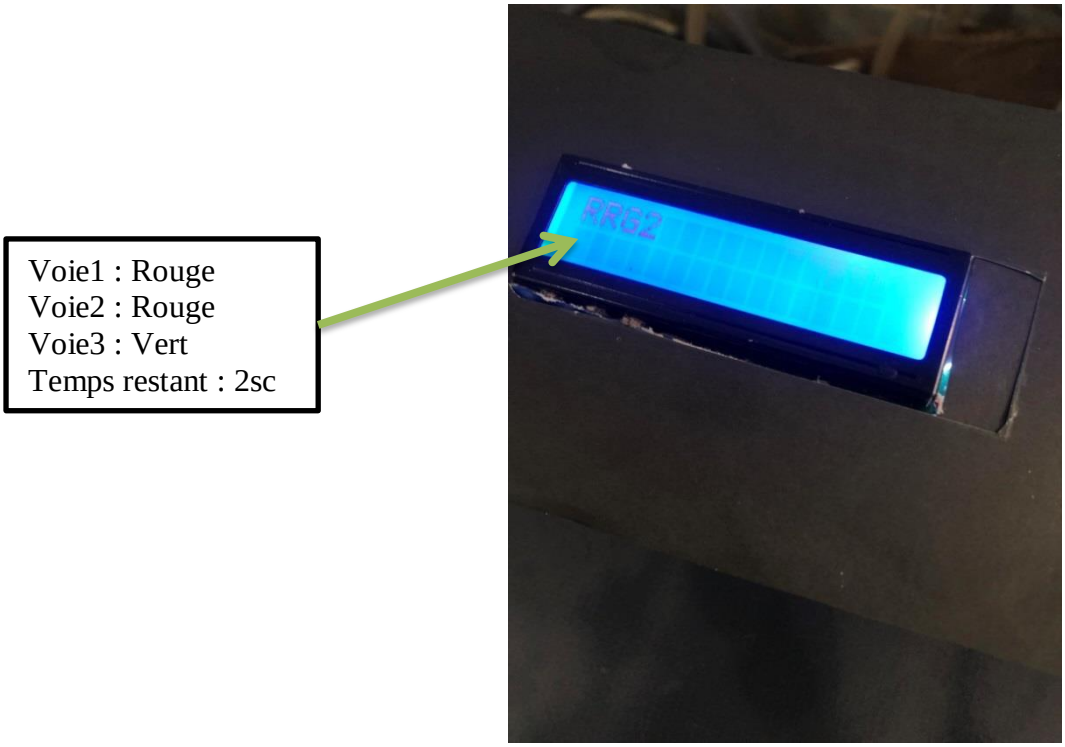


Figure 57: Affichage de l'état + temps

9.2. Cas d'un véhicule prioritaire (ambulance)



Figure 58: Ambulance détectée



Figure 59: Affichage d'urgence

9.3. Cas d'un véhicule intelligent



Figure 60: Envoi du feu rouge



Figure 61: Envoi du feu jaune

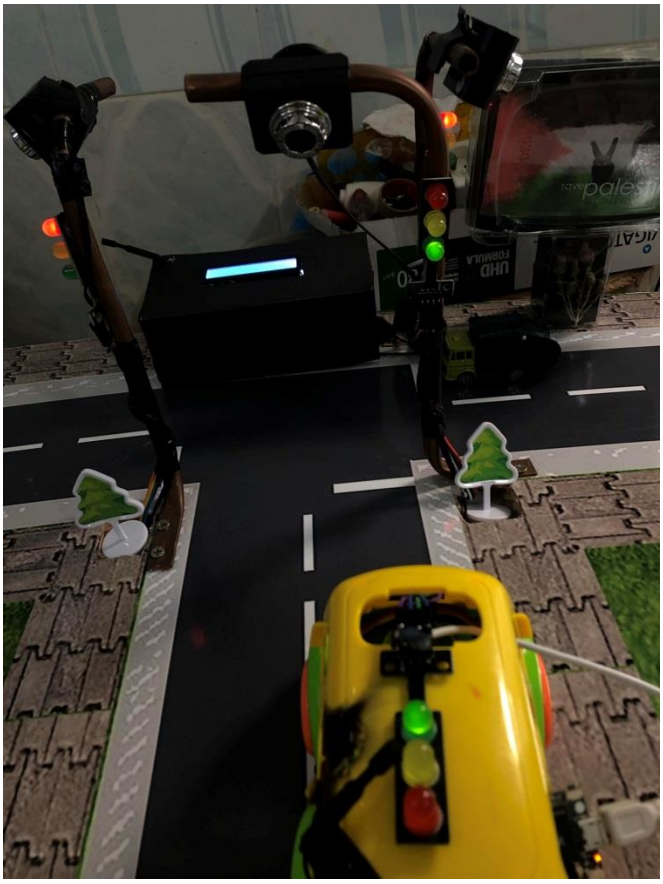


Figure 62: Envoi du feu vert

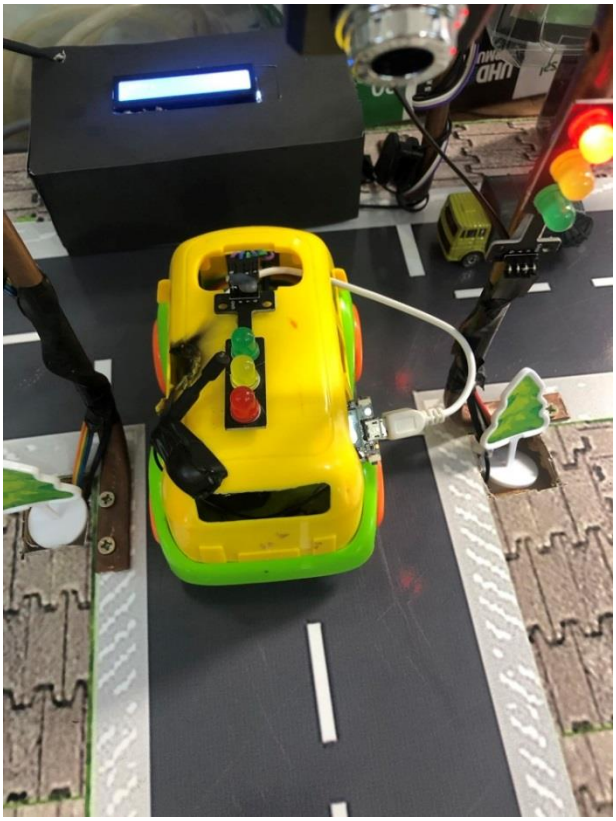


Figure 63: Arrivée à l'intersection

Conclusion

En conclusion, ce chapitre a méticuleusement décrit la conception et les composants de notre système de contrôle de trafic intelligent. En explorant les différentes intersections et technologies impliquées, nous avons mis en évidence les considérations et décisions qui sous-tendent notre système. Cet examen détaillé démontre non seulement la sophistication et la complexité de la solution de gestion du trafic, mais souligne également son potentiel à améliorer l'efficacité et la sécurité du trafic. Grâce à un placement stratégique des caméras et à des algorithmes avancés, notre système est prêt à gérer efficacement les flux de trafic urbains, assurant une expérience de transit fluide et sécurisée pour tous les usagers de la route

Conclusion générale

Au terme de ce projet, nous avons démontré la faisabilité et l'efficacité d'un système intelligent de gestion du trafic, intégrant des technologies avancées telles que l'IoT, l'intelligence artificielle et la communication sans fil LoRa. Ce système permet une gestion dynamique et adaptative des feux de signalisation, optimisant ainsi la fluidité du trafic et la sécurité routière.

Les différentes étapes, de la conception à la réalisation, ont été détaillées dans les chapitres précédents, mettant en évidence les défis techniques et les solutions apportées. L'intégration réussie des composants matériels et logiciels, ainsi que l'entraînement efficace du modèle YOLOv8 pour la détection d'objets, ont permis de créer un prototype fonctionnel capable de gérer les intersections de manière intelligente.

Les résultats obtenus montrent que notre système est capable de réduire significativement les temps d'attente aux carrefours, de prioriser les véhicules d'urgence et de minimiser les risques d'accidents. De plus, la capacité du système à collecter et analyser des données en temps réel ouvre la voie à une planification proactive des infrastructures urbaines, contribuant ainsi à une meilleure gestion de la mobilité urbaine.

En conclusion, ce projet illustre le potentiel des technologies intelligentes pour transformer la gestion du trafic urbain. Il pose les bases pour des développements futurs visant à améliorer encore la précision et l'efficacité des systèmes de gestion de trafic, tout en s'intégrant harmonieusement dans les écosystèmes des villes intelligentes, ouvrant ainsi de nouvelles perspectives pour l'avenir.

Nétographie

[1] https://moome.io/fr/accueil/ - consulte « 14/04/2024 »

[2] https://www.scats.com.au/home - consulte « 14/04/2024 »

[3] https://trlsoftware.com/products/traffic-control/scoot/ - consulte « 14/04/2024 »

[4] https://www.futura-sciences.co - consulte « 14/04/2024 »

[5] https://www.connectwave.fr - consulte « 14/04/2024 »

[6] https://www.mokosmart.com - consulte « 14/04/2024 »

[7] https://www.bable-smartcities.eu/home.html - consulte « 14/04/2024 »

[8] https://www.raspberrypi.org/ - consulte « 15/04/2024 »

[9] https://en.wikipedia.org/wiki/Raspberry_Pi - consulte « 15/04/2024 »

[10] https://www.digikey.com/ - consulte « 15/04/2024 »

[11] https://www.mouser.com/ - consulte « 15/04/2024 »

[12] https://www.sparkfun.com/ - consulte « 15/04/2024 »

[13] https://www.adafruit.com/ - consulte « 15/04/2024 »

[14] https://lora-alliance.org/ - consulte « 15/04/2024 »

[15] https://mqtt.org/ - consulte « 15/04/2024 »

[16] https://heltec.org/ - consulte « 15/04/2024 »

[17] https://github.com/ - consulte « 15/04/2024 »

[18] https://academy.networkchuck.com/ - consulte « 16/04/2024 »

[19] https://core-electronics.com.au/guides/object-identify-raspberry-pi/ - consulte « 16/04/2024 »

[20] https://pysource.com/2022/01/10/build-your-object-detection-software-crash-course/ - consulte « 16/04/2024 »

[21] https://iotline.eu/spi-with-multiple-devices-attached-arduino-heltec-wifi-lora-32-and-rfid-rc522/ - consulte « 25/04/2024 »

[22] https://pubsubclient.knolleary.net/api - consulte « 25/04/2024 »

[23] https://code.visualstudio.com/ - consulte « 1/05/2024 »

[24] https://platformio.org/ - consulte « 1/05/2024 »

[25] https://www.arduino.cc/en/software - consulte « 1/05/2024 »

[26] https://thonny.org/ - consulte « 16/04/2024 »

[27] https://www.microsoft.com/fr-fr/windows/paint - consulte « 20/04/2024 »

[28] https://fritzing.org/ - consulte « 05/05/2024 »

[29] https://github.com/HumanSignal/labelImg - consulte « 19/04/2024 »

[30] https://www.python.org/ - consulte « 13/05/2024 »

[31] https://docs.python.org/3/library/ - consulte « 15/04/2024 »

[32] https://gpiozero.readthedocs.io/en/latest/ - consulte « 14/04/2024 »

ANNEXES

1. Logiciels

Visual Studio Code



Figure 64: VS Code

Visual Studio Code (VS Code) est un éditeur de code source gratuit et open-source développé par Microsoft. Lancé en 2015, il est apprécié pour sa légèreté, sa rapidité et ses nombreuses fonctionnalités, notamment la coloration syntaxique, l'autocomplétions, le débogage intégré et la gestion de git. Grâce à une vaste collection d'extensions, VS Code peut être personnalisé pour prendre en charge une multitude de langages de programmation et de Framework. Compatible avec Windows, MacOs et Linux, c'est un outil populaire parmi les développeurs.[23]

PlatformIO



Figure 65: PlatformIO

PlatformIO est un environnement de développement intégré (IDE) open-source conçu pour le développement de logiciels embarqués. Il est compatible avec de nombreux microcontrôleurs et cartes de développement, tels que Arduino, ESP32 et STM32. PlatformIO offre des fonctionnalités telles que la gestion des bibliothèques, le débogage avancé, l'intégration avec des éditeurs comme VS Code, et un flux

de travail multiplateforme. Il simplifie la configuration des projets et améliore la productivité des développeurs grâce à ses outils et extensions robustes. [24]

Arduino IDE



Figure 66: Arduino IDE

Arduino IDE (Integrated Development Environment) est un logiciel open-source utilisé pour écrire, compiler et téléverser des programmes sur des cartes de développement Arduino. Conçu pour être simple et convivial, il prend en charge les langages C et C++ avec des bibliothèques Arduino spécifiques pour simplifier la programmation. L'IDE offre une interface intuitive, la gestion des bibliothèques et un moniteur série pour la communication en temps réel avec les cartes Arduino. Disponible sur Windows, MacOs et Linux, l'Arduino IDE est idéal pour les débutants et les développeurs expérimentés dans le domaine des projets électroniques et embarqués. [25]

Thonny

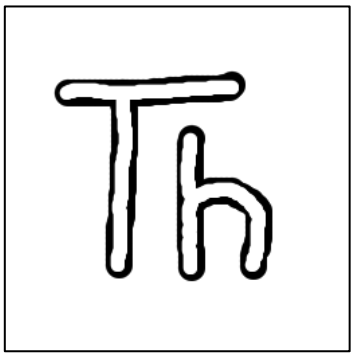


Figure 67: Thonny

Thonny est un environnement de développement intégré (IDE) pour Python. Il est conçu pour être simple et convivial, idéal pour les débutants en programmation Python. Thonny offre une interface intuitive avec des fonctionnalités telles que l'édition de code, le débogage et la gestion des fichiers. C'est un outil pratique pour apprendre et pratiquer la programmation Python. [26]

MS Paint



Figure 68: MS Paint

Logiciel de dessin basique de Microsoft, inclus dans Windows, idéal pour des tâches simples de création et de modification d'images. [27]

Fritzing



Figure 69: Fritzing

Fritzing est un logiciel open-source destiné à la conception de circuits électroniques. Il offre une interface conviviale et intuitive, permettant aux utilisateurs de créer des schémas électriques, des circuits imprimés (PCB) et des dessins de prototype de manière facile et visuelle. Fritzing est largement utilisé par les amateurs, les étudiants et les professionnels pour prototyper et documenter des projets électroniques. [28]

LabelImg



Figure 70: LabelImg

LabelImg est un outil open-source populaire utilisé pour annoter des images en vue de l'apprentissage automatique, en particulier pour la création de jeux de données pour la détection d'objets. Il permet aux

utilisateurs de marquer des objets dans des images en dessinant des boîtes englobantes autour d'eux et de leur attribuer des étiquettes. LabelImg est apprécié pour sa simplicité d'utilisation, sa flexibilité et sa compatibilité avec divers formats de fichiers d'images. [29]

2. Langages de programmation

Python



Figure 71: Python

Python est un langage de programmation interprété et polyvalent, créé en 1991 par Guido van Rossum. Apprécié pour sa syntaxe claire, il facilite l'apprentissage et le développement rapide. Il prend en charge divers paradigmes et dispose d'une vaste bibliothèque standard, le rendant idéal pour le développement web, l'analyse de données, l'IA et plus encore. Portable et fonctionnant sur de nombreuses plateformes, Python est très populaire. [30]

Arduino



Figure 72: C++/Arduino

Le langage Arduino est une version simplifiée de C/C++ utilisée pour programmer les cartes Arduino. Il offre une syntaxe simplifiée et des bibliothèques prêtes à l'emploi pour simplifier le développement de projets électroniques. [25]

3. Librairies Python

Ultralytics YOLO

Ultralytics YOLO est une bibliothèque populaire pour la détection d'objets en temps réel. Nous l'avons utilisée pour entraîner et déployer notre modèle de détection de véhicules prioritaires, notamment les ambulances et les voitures de police. Cette bibliothèque nous a permis d'obtenir des résultats précis et rapides, essentiels pour la détection en temps réel dans notre système de contrôle de trafic. [31]

OS (Operating System Interface)

La bibliothèque OS nous a fourni des fonctionnalités pour interagir avec le système d'exploitation sous-jacent de manière portable et efficace. Nous l'avons utilisée pour gérer les opérations liées au système de fichiers, telles que la création de répertoires, la manipulation des chemins de fichiers et l'exécution de commandes système. [31]

GPIOZero

GPIOZero est une bibliothèque Python simple et élégante pour contrôler les broches GPIO (General Purpose Input/Output) sur les cartes Raspberry Pi. Nous l'avons utilisée pour interagir avec les broches GPIO et contrôler les composants matériels tels que les LEDs de signalisation du trafic et les capteurs. [32]

Time

La bibliothèque Time nous a fourni des fonctions pour mesurer et gérer le temps dans notre application. Nous l'avons utilisée pour synchroniser les événements, mettre en œuvre des délais et contrôler le timing des opérations critiques dans notre système. [31]

CV2 (OpenCV)

OpenCV (Open Source Computer Vision Library) est une bibliothèque open-source de vision par ordinateur et de traitement d'images. Nous l'avons utilisée pour capturer et traiter les flux vidéo des caméras, ainsi que pour effectuer diverses opérations de traitement d'images telles que la détection d'objets et la reconnaissance de formes. [31]

NumPy

NumPy est une bibliothèque Python largement utilisée pour le calcul numérique. Nous l'avons utilisée pour effectuer des opérations de traitement d'images efficaces et pour manipuler des tableaux de données multidimensionnels dans le cadre de nos algorithmes de détection d'objets. [31]

Paho MQTT Client

Le client Paho MQTT est une bibliothèque Python qui implémente le protocole MQTT (Message Queuing Telemetry Transport). Nous l'avons utilisée pour faciliter la communication entre les différents composants de notre système de contrôle de trafic intelligent, permettant l'échange de données en temps réel de manière fiable et sécurisée. [31]

Multiprocessing

La bibliothèque Multiprocessing nous a permis de créer et de gérer plusieurs processus en parallèle, augmentant ainsi la performance et l'efficacité de notre application. Nous l'avons utilisée pour exécuter des tâches intensives sur différents cœurs de processeur, réduisant les temps de latence et améliorant la répartition de la charge de travail. [31]

Threading

La bibliothèque Threading nous a permis de créer des threads légers et concurrents dans notre application. Nous l'avons utilisée pour exécuter des tâches en parallèle, améliorant ainsi les performances et la réactivité de notre système. [31]

Random

La bibliothèque Random nous a fourni des fonctions pour générer des nombres aléatoires dans notre application. Nous l'avons utilisée pour simuler des événements aléatoires, tester des scénarios variés et garantir la robustesse de notre système face à des situations imprévues. [31]

4. Librairies Arduino

SPI

Cette bibliothèque fournit des fonctions pour communiquer avec des dispositifs en utilisant le protocole d'Interface Périphérique Série (SPI). Elle est couramment utilisée pour interagir avec des capteurs, des écrans et d'autres périphériques. [25]

LoRa

LoRa est une technologie de communication sans fil à longue portée et basse consommation. Cette bibliothèque vous permet d'utiliser des modules LoRa pour transmettre et recevoir des données sur de longues distances avec une faible consommation d'énergie. [25]

Wi-Fi

Cette bibliothèque permet la communication avec les réseaux Wi-Fi. Elle fournit des fonctions pour se connecter aux réseaux Wi-Fi, configurer les paramètres réseau et échanger des données via des connexions Wi-Fi. [25]

Wire

Wire est un protocole de communication utilisé pour connecter plusieurs dispositifs dans une configuration maître-esclave. Cette bibliothèque fournit des fonctions pour implémenter la communication I2C, vous permettant de connecter et de communiquer avec des dispositifs compatibles I2C tels que des capteurs et des écrans. [25]

LiquidCrystal_I2C

Cette bibliothèque est une extension de la bibliothèque LiquidCrystal et est spécifiquement conçue pour contrôler des affichages LCD en utilisant le protocole I2C. Elle simplifie le processus d'interaction avec des affichages LCD compatibles I2C. [25]

PubSubClient

PubSubClient est une bibliothèque pour le protocole de communication MQTT (Message Queuing Telemetry Transport). Elle permet aux dispositifs de publier et de souscrire à des sujets sur un courtier MQTT, facilitant la communication entre les dispositifs dans les applications Internet des Objets (IoT). [25]

MFRC522

La bibliothèque "mfrc522.h" est utilisée pour interagir avec le module MFRC522, un lecteur de cartes RFID. Elle permet de lire et écrire des données sur des cartes RFID compatibles avec le standard MIFARE, ainsi que de détecter les cartes RFID à proximité. [25]

ملخص

في المدن الذكية، تعمل أنظمة إدارة التقاطعات المرورية الذكية على تعزيز السلامة من خلال تكييف إشارات المرور، وإعطاء الأولوية لخدمات الطوارئ وتقليل أوقات الانتظار، مع تحسين كفاءة الطاقة وتسهيل الإدارة الاستباقية بشأن الحوادث. فهي تساعد في إنشاء أنظمة بيئية حضرية متماسكة ومُحسّنة، وبالتالي تلعب دورًا حاسمًا في بناء مدن مستدامة ومتصلة للمستقبل.

يتكون مشروعنا من تطوير نظام ذكي لإدارة التقاطعات المرورية يعتمد على تقنيات إنترنت الأشياء والتعرف على السيارات ذات الأولوية بواسطة كاميرات المراقبة. يرتكز مشروعنا على العديد من التقنيات المتقدمة مثل إنترنت الأشياء واتصالات LoRa وأجهزة الكومبيوتر صغيرة الحجم مثل Raspberry Pi وESP32 من Heletc .

Abstract

In smart cities, intelligent intersection management systems enhance safety by adapting traffic lights, prioritizing emergency services and reducing wait times, while improving energy efficiency and facilitating management proactive about incidents. They help create coherent and optimized urban ecosystems, thus playing a crucial role in building sustainable and connected cities for the future.

Our project consists of developing an intelligent intersection management system based on Internet of Things technologies and the recognition of priority automobiles by surveillance cameras. Our projects focus on several advanced technologies such as the Internet of Things, LoRa communication and Raspberry Pi and ESP32 boards from Heletc.

Résumé

Dans les villes intelligentes, les systèmes de gestion intelligent de carrefour renforcent la sécurité en adaptant les feux de circulation, en priorisant les services d'urgence et en réduisant les temps d'attente, tout en améliorant l'efficacité énergétique et en facilitant la gestion proactive des incidents. Ils contribuent à créer des écosystèmes urbains cohérents et optimisés, jouant ainsi un rôle crucial dans la construction de villes durables et connectées pour l'avenir.

Notre projet consiste à développer un système de gestion intelligent de carrefour qui s’appui sur les technologies de l’Internet des Objets et la reconnaissance des automobiles prioritaires par caméras de surveillances. Notre projet s’articule sur plusieurs technologies avancées telles que l’Internet des Objets, la communication LoRa et les cartes Raspberry Pi et ESP32 de Heletc.